

POLITECNICO DI TORINO

**Corso di Laurea
in Matematica per l'Ingegneria**

Colucci Giorgia, matricola s323617

Matematica per l'Intelligenza Artificiale

Dataset: Student Performance (corso di Portoghese)

**Studio del dataset *Student Performance*
mediante tecniche di Supervised ed Unsupervised Learning**



Indice

1	Descrizione del Dataset	3
2	Pre-elaborazione del Dataset e Fondamenti Teorici	5
2.1	La maledizione della dimensionalità (<i>curse of dimensionality</i>)	5
2.2	Perché standardizzare?	6
2.3	Strategia di Encoding	6
3	Principal Component Analysis (PCA)	7
3.1	Motivazioni	7
3.2	Fondamenti Teorici	7
3.3	Interpretazione Geometrica	9
3.4	Obiettivo nel dataset <i>Student Performance</i>	10
3.5	Analisi della Varianza Spiegata (Scree Plot)	10
3.6	Interpretazione dei Risultati: perché serve la soglia di varianza	11
3.7	Score Plot e Loading Graph	12
3.8	Osservazioni sui Risultati della PCA	13
3.9	Score Plot con Target Binario	14
4	Preparazione dei Dati per la Classificazione	14
5	Fisher / Multiple Discriminant Analysis (FDA / MDA)	16
5.1	Fondamenti Teorici	16
5.2	Proiezione MDA con classe dedicata e confronto con la PCA	18
5.3	Caso Binario: FDA a un asse e confronto binario / multiclasse	18
5.4	FDA e MDA sui dati compressi dalla PCA	19
5.5	LDA e QDA come classificatori; confronto PCA / MDA / LDA	20
6	Support Vector Machine (SVM)	23
6.1	Fondamenti Teorici	23
6.2	Margine Rigido vs Margine Morbido (Hard vs Soft Margin)	24
6.3	Soft SVM sui dati compressi dalla PCA	25
6.4	Visualizzazione del confine di decisione (proiezione LDA 2D)	25
6.5	Nota sulle SVM non lineari e multiclasse	26

7	K-Nearest Neighbors (K-NN)	26
7.1	Fondamenti Teorici	26
7.2	K-NN sui dati compressi dalla PCA	28
8	Multi-Layer Perceptron (MLP)	28
8.1	Fondamenti Teorici	28
8.2	Selezione degli iperparametri (Grid Search + Cross-Validation)	29
9	Confronto dei Modelli	30
9.1	Bilanciamento delle Classi e Metriche Robuste	31
9.2	Visualizzazione degli Errori sulla Proiezione	32
9.3	Validazione robusta: Cross-Validation (k-fold)	32
10	Conclusioni	34
11	Bibliografia	36

1 Descrizione del Dataset

Il dataset *Student Performance* (sottoinsieme Portoghese) contiene informazioni demografiche, sociali e scolastiche relative a studenti di un corso di lingua portoghese in due scuole secondarie portoghesi (Gabriel Pereira e Mousinho da Silveira, a.s. 2005–2006). Il dataset contiene **649 studenti** descritti da **33 attributi**, senza valori mancanti.

Scelta del corso. Il repository UCI fornisce due dataset distinti — Matematica (395 studenti) e Portoghese (649) — che gli autori trattano *separatamente*. Si è scelto il **corso di Portoghese** perché più numeroso (649 vs 395 osservazioni): più dati garantiscono stime più stabili, una suddivisione train/test più affidabile e classi meno esposte a fluttuazioni campionarie. Concentrarsi su una singola materia evita inoltre di mescolare due popolazioni con dinamiche di valutazione diverse.

Il problema considerato consiste nella previsione del voto finale G3, espresso su una scala da 0 a 20. In questo studio G3 viene **discretizzato** in tre fasce di rendimento (§4) e utilizzato sia per definire le classi dei modelli di classificazione sia per evidenziare la struttura dei dati negli *score plot* della PCA.

Nota metodologica (target leakage). Le feature G1 e G2 (voti del 1° e 2° trimestre) sono quasi perfettamente correlate con G3. La documentazione ufficiale del dataset (Cortez & Silva, 2008; UCI Machine Learning Repository) segnala esplicitamente che «G3 ha una forte correlazione con G2 e G1, poiché G3 è il voto finale (3° periodo) mentre G1 e G2 corrispondono ai voti del 1° e 2° periodo». Il dato è confermato dalla **correlazione di Pearson** calcolata sul corso di Portoghese (649 osservazioni): $r_{G1,G3} \approx 0.83$ e $r_{G2,G3} \approx 0.92$ (abbiamo quindi forte correlazione perché i valori sono vicini all'estremo 1). Includerle renderebbe il problema banale e maschererebbe il contributo delle variabili socio-comportamentali: per questo G1 e G2 vengono **escluse** da tutte le analisi.

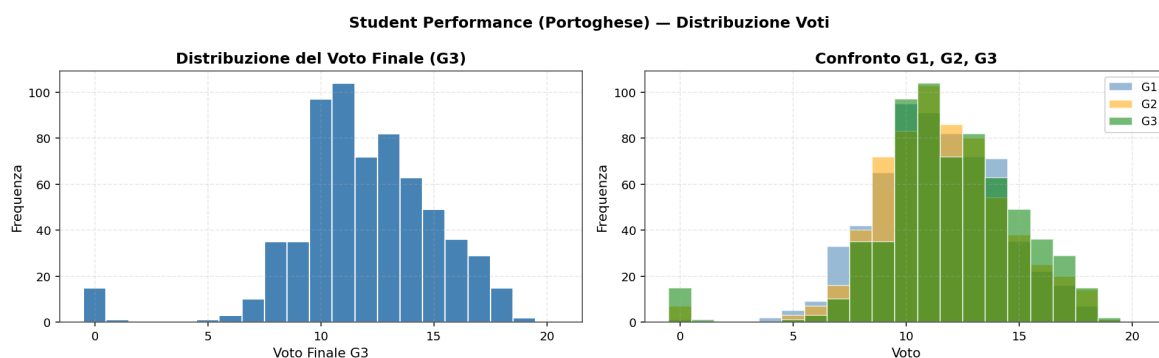


Figura 1: Distribuzione del voto finale G3 e confronto con G1, G2.

Tabella 1: Elenco completo degli attributi del dataset.

N.	Attributo	Tipo	Valori	Descrizione
1	school	Binario	GP / MS	Scuola di appartenenza
2	sex	Binario	F / M	Sesso
3	age	Numerico	15–22	Età
4	address	Binario	U / R	Residenza (urbana / rurale)
5	famsize	Binario	LE3 / GT3	Nucleo familiare (≤ 3 / > 3)

Tabella 1 – continua

N.	Attributo	Tipo	Valori	Descrizione
6	Pstatus	Binario	T / A	Convivenza genitori (insieme / separati)
7	Medu	Ordinale	0–4	Istruzione madre (0=nessuna, 4=lau-rea)
8	Fedu	Ordinale	0–4	Istruzione padre
9	Mjob	Nominale	5 cat.	Professione della madre
10	Fjob	Nominale	5 cat.	Professione del padre
11	reason	Nominale	4 cat.	Motivo di scelta della scuola
12	guardian	Nominale	3 cat.	Tutore legale
13	traveltime	Ordinale	1–4	Tempo casa–scuola
14	studytime	Ordinale	1–4	Ore di studio settimanali
15	failures	Numerico	0–4	Bocciature pregresse
16	schoolsup	Binario	yes/no	Supporto educativo scolastico
17	famsup	Binario	yes/no	Supporto educativo familiare
18	paid	Binario	yes/no	Lezioni private a pagamento
19	activities	Binario	yes/no	Attività extracurricolari
20	nursery	Binario	yes/no	Frequenzazione asilo nido
21	higher	Binario	yes/no	Aspirazione a istruzione superiore
22	internet	Binario	yes/no	Accesso a internet a casa
23	romantic	Binario	yes/no	Relazione sentimentale in corso
24	famrel	Ordinale	1–5	Qualità relazioni familiari
25	freetime	Ordinale	1–5	Tempo libero dopo la scuola
26	goout	Ordinale	1–5	Frequenza uscite con amici
27	Dalc	Ordinale	1–5	Consumo alcol giorni feriali
28	Walc	Ordinale	1–5	Consumo alcol fine settimana
29	health	Ordinale	1–5	Stato di salute percepito
30	absences	Numerico	0–93	Numero di assenze
31	G1	Numerico	0–20	Voto 1° trimestre (escluso)
32	G2	Numerico	0–20	Voto 2° trimestre (escluso)
33	G3	Numerico	0–20	Voto finale (target, discretizzato)

Categorizzazione delle Variabili

Numeriche / Ordinali (già numeriche nel CSV): age, Medu, Fedu, traveltime, studytime, failures, famrel, freetime, goout, Dalc, Walc, health, absences.

Binarie (Label Encoding $\rightarrow \{0,1\}$): school, sex, address, famsize, Pstatus, schoolsup, famsup, paid, activities, nursery, higher, internet, romantic.

Nominali multi-classe (One-Hot Encoding): Mjob {teacher, health, services, at_home, other}, Fjob {teacher, health, services, at_home, other}, reason {home, reputation, course, other},

guardian {mother, father, other}.

Escluse (target leakage): G1, G2. **Target (discretizzato):** G3.

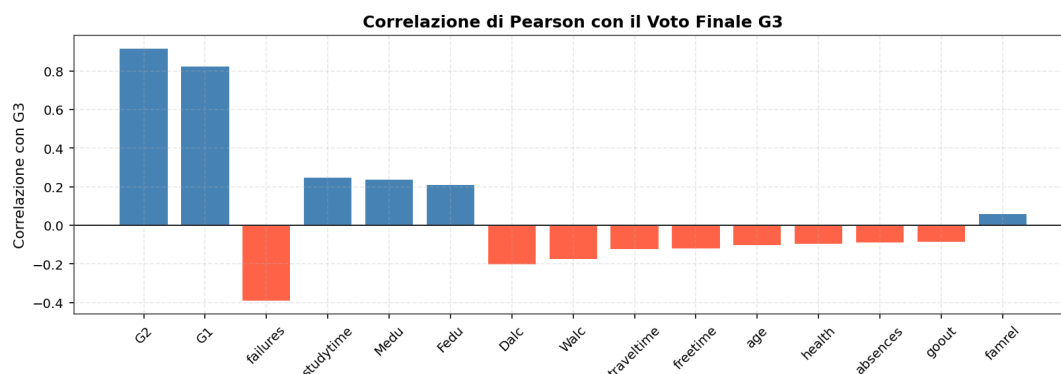


Figura 2: Correlazione di Pearson di ciascuna variabile numerica con il voto finale G3. I voti intermedi G1 e G2 dominano nettamente — da cui la loro esclusione per evitare il *target leakage* — mentre tra i predittori restanti emergono failures (negativo) e studytime, Medu, Fedu (positivi).

2 Pre-elaborazione del Dataset e Fondamenti Teorici

2.1 La maledizione della dimensionalità (*curse of dimensionality*)

Dopo la codifica delle variabili categoriche il dataset vivrà in uno spazio a $p = 43$ dimensioni (le 43 feature ottenute dopo l’encoding; il target G3 non è una dimensione di questo spazio). All’aumentare di p lo spazio si “svuota”: il volume cresce esponenzialmente e, a parità di campioni, i dati diventano sempre più **sparsi**. Un modo quantitativo di vederlo è la **distanza mediana** dall’origine al punto più vicino tra N campioni distribuiti uniformemente nella palla unitaria di $\mathbb{R}^p$:

$$d(N, p) = \left(1 - \frac{1}{2}^{1/N}\right)^{1/p}.$$

Già con $N = 500$ e $p = 10$ si ottiene $d \approx 0.52$: il vicino più prossimo è *più vicino al bordo che al centro*, così che predire localmente significa **estrapolare** dai margini anziché interpolare. Le conseguenze rilevanti per questo studio sono:

1. le **distanze euclidee** si concentrano (il rapporto tra distanza massima e minima tende a 1), erodendo il potere discriminante di metodi come il K-NN (§7);
2. le stime con molti parametri — ad esempio una covarianza piena per classe nella QDA (§5) — diventano instabili;
3. nozioni di densità e vicinanza perdono significato.

È questo fenomeno a motivare l’intero percorso di **riduzione della dimensionalità** (PCA, §3; FDA/MDA, §5) che segue.

La PCA richiede feature **numeriche** con **scala comparabile**. Il dataset presenta però attributi con scale molto diverse: l’età varia da 15 a 22, le assenze arrivano fino a oltre 90, mentre molte feature ordinali (es. la qualità delle relazioni familiari) vivono su una scala 1–5. Inoltre 17 colonne sono testuali e vanno codificate numericamente.

2.2 Perché standardizzare?

La PCA ricerca le direzioni lungo cui i dati si **disperdono maggiormente** (massima varianza). Se si applicasse la trasformazione direttamente alla matrice di **covarianza** dei dati grezzi, le feature con range numerico maggiore (es. absences) dominerebbero la varianza totale, oscurando il contributo delle altre variabili. Si procede quindi con la **standardizzazione** (z-score): a ogni colonna si sottrae la media e si divide per la deviazione standard, così che ogni feature abbia media 0 e varianza 1.

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j},$$

dove si ha

$$\mu_j = \frac{1}{m} \sum_{i=1}^m x_{ij}.$$

e

$$\sigma_j = \sqrt{\frac{1}{m-1} \sum_{i=1}^m \left(x_j^{(i)} - \mu_j\right)^2}, \quad j = 1, \dots, n.$$

Matematicamente, applicare la PCA ai dati standardizzati equivale ad estrarre autovalori e autovettori dalla **matrice di correlazione** anziché da quella di covarianza, garantendo a ogni variabile lo stesso peso iniziale.

2.3 Strategia di Encoding

1. **Target leakage.** I voti intermedi G1, G2 sono rimossi (correlazione con G3 quasi unitaria). Il target G3 viene discretizzato in 3 classi e conservato solo come etichetta.
2. **Label Encoding** per le 13 variabili binarie → ogni categoria mappata in $\{0, 1\}$.
3. **One-Hot Encoding** per le 4 variabili nominali (Mjob, Fjob, reason, guardian) → variabili dummy binarie.
4. **StandardScaler** su tutte le feature risultanti (media 0, varianza 1) prima della PCA.

Prima di codificare osserviamo la **matrice di correlazione** delle variabili numeriche grezze (Figura 3): essa mostra la presenza di ridondanza informativa (gruppi di variabili correlate) che giustifica l'uso della PCA.

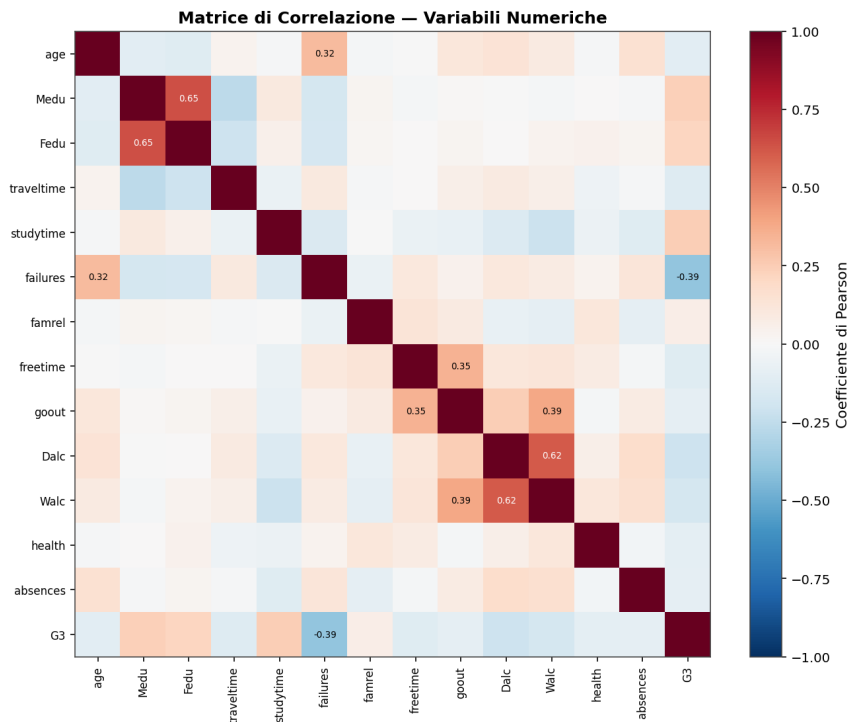


Figura 3: Matrice di correlazione delle variabili numeriche grezze: evidenzia la ridondanza che motiva la PCA.

3 Principal Component Analysis (PCA)

3.1 Motivazioni

Dopo la codifica delle variabili categoriche il numero di feature è elevato ($p = 43$) e molte risultano correlate (istruzione di madre e padre; studio e aspirazione all'università; bocciature e rendimento). Questa **ridondanza** suggerisce che il fenomeno possa essere descritto da poche variabili latenti.

La **Principal Component Analysis (PCA)** è una tecnica *non supervisionata* che costruisce nuove variabili sintetiche — le *componenti principali* — come combinazioni lineari ortogonali delle feature originali, ordinate per varianza spiegata decrescente. L'obiettivo è una rappresentazione più compatta che conservi la maggior parte dell'informazione riducendo il rumore.

3.2 Fondamenti Teorici

Sia $X \in \mathbb{R}^{N \times p}$ la matrice dei dati (N osservazioni, p feature). Dopo la standardizzazione

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}$$

si ottiene la matrice Z , a media nulla su ogni colonna. La PCA cerca combinazioni lineari delle feature — le componenti principali — a **varianza massima** e mutuamente **scorrelate**. Lo stesso insieme di direzioni si può ottenere da due principi apparentemente diversi — **massimizzare la varianza proiettata** e **minimizzare l'errore di ricostruzione** — che si dimostrano *equivalenti*: entrambi conducono al medesimo problema agli autovalori.

3.2.1 Formulazione a massima varianza

Proiettando i dati sul versore $v \in \mathbb{R}^p$, $\|v\| = 1$, si ottiene la componente PC $= Zv$. Poiché Z è centrata, anche la proiezione ha media nulla e la sua varianza campionaria è

$$\text{Var}(Zv) = \frac{1}{N-1} \|Zv\|^2 = \frac{1}{N-1} v^\top Z^\top Z v = v^\top R v,$$

dove $R = \frac{1}{N-1} Z^\top Z$ è la **matrice di correlazione** (la covarianza dei dati standardizzati). La prima componente è la direzione che rende massima questa varianza:

$$v_1 = \arg \max_{\|v\|=1} v^\top R v.$$

Imponendo il vincolo $\|v\|^2 = 1$ con un moltiplicatore di Lagrange λ e annullando il gradiente della lagrangiana $\mathcal{L}(v, \lambda) = v^\top R v - \lambda(v^\top v - 1)$,

$$\nabla_v \mathcal{L} = 2Rv - 2\lambda v = 0 \implies Rv = \lambda v,$$

si vede che i punti stazionari sono gli **autovettori** di R , e che la varianza lungo ciascuno vale $v_i^\top R v_i = \lambda_i$: massimizzare la varianza equivale a scegliere l'autovettore con autovalore più grande. La k -esima componente massimizza la varianza residua restando ortogonale alle precedenti, e coincide con il k -esimo autovettore. Si ottiene così la base ordinata

$$R v_i = \lambda_i v_i, \quad \lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p \geq 0,$$

con v_i **direzioni principali** (i *loading vector*) e λ_i pari alla **varianza spiegata** dalla i -esima componente. Le componenti $\text{PC}_k = Zv_k$ risultano quindi **ortogonali**, **scorrelate** e **ordinate** per varianza decrescente.

3.2.2 Formulazione a minimo errore di ricostruzione

In alternativa si cerca il sottospazio k -dimensionale che **rappresenta meglio i dati**. Sia $\{e_1, \dots, e_k\}$ una base ortonormale del sottospazio: ogni punto centrato z_j vi si approssima con $\sum_{i=1}^k \alpha_{ji} e_i$, e si minimizza l'errore quadratico totale (la somma dei quadrati delle distanze dei punti dalla loro proiezione)

$$J(e_1, \dots, e_k; \alpha) = \sum_{j=1}^N \left\| z_j - \sum_{i=1}^k \alpha_{ji} e_i \right\|^2.$$

Annullando $\partial J / \partial \alpha_{ji}$ si ottiene subito il coefficiente ottimo $\alpha_{ji} = z_j^\top e_i$ (la **proiezione ortogonale** del dato sull'asse). Sostituendolo, l'errore si riscrive

$$J = \sum_{j=1}^N \|z_j\|^2 - \sum_{i=1}^k e_i^\top S e_i, \quad S = \sum_{j=1}^N z_j z_j^\top = Z^\top Z,$$

dove S è la **matrice di scatter**, pari a $(N-1)$ volte la covarianza campionaria (quindi $S = (N-1)R$). Poiché $\sum_j \|z_j\|^2$ è costante, **minimizzare J equivale a massimizzare $\sum_i e_i^\top S e_i$** sotto il vincolo di ortonormalità $e_i^\top e_i = 1$. Introducendo nuovamente i moltiplicatori di Lagrange si ritrova un problema agli autovalori,

$$S e_m = \lambda_m^S e_m,$$

la cui soluzione ottima sono i k autovettori di S — *identici* a quelli di R , dato che $S = (N-1)R$ — associati ai k autovalori maggiori. L'errore minimo vale allora la somma degli autovalori **scartati**,

$$J_{\min} = \sum_{i>k} \lambda_i^S = (N-1) \sum_{i>k} \lambda_i,$$

ossia, nella stessa normalizzazione della varianza, $\frac{1}{N-1} J_{\min} = \sum_{i>k} \lambda_i$.

3.2.3 Equivalenza e rappresentazione ridotta

Le due formulazioni portano agli **stessi autovettori**: la varianza *trattenuta* $\sum_{i \leq k} \lambda_i$ e l'errore di ricostruzione *medio* $\sum_{i > k} \lambda_i$ sono complementari, con somma pari alla varianza totale $\sum_i \lambda_i = \text{tr}(R) = p$ (per dati standardizzati ogni feature contribuisce 1). Massimizzare la prima equivale a minimizzare il secondo: è il motivo per cui un'unica decomposizione spettrale risolve entrambi i problemi.

Raccogliendo i primi m autovettori per colonne in $B = [v_1 \cdots v_m] \in \mathbb{R}^{p \times m}$, la rappresentazione ridotta di un dato è $y_i = B^\top z_i \in \mathbb{R}^m$ (un cambio di coordinate nella nuova base ortonormale, $y = B^\top z$) e la **ricostruzione** approssimata è $\tilde{z}_i = B B^\top z_i$, con errore quadratico medio pari alla somma degli autovalori scartati, $\sum_{i > m} \lambda_i$.

3.2.4 La SVD e il legame con la PCA

Lettura geometrica. Una trasformazione lineare non singolare manda sempre l'ipersfera unitaria in un **iperellissoide**. La **decomposizione ai valori singolari (SVD)** ne individua gli assi: per *qualunque* matrice $Z \in \mathbb{R}^{N \times p}$ esiste la fattorizzazione

$$Z = U \Sigma V^\top,$$

con $U \in \mathbb{R}^{N \times N}$ e $V \in \mathbb{R}^{p \times p}$ **ortogonali** e Σ diagonale dei **valori singolari** $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$. Le colonne di U sono i **vettori singolari sinistri** (gli assi dell'ellissoide), quelle di V i **vettori singolari destri** (le loro preimmagini); vale la relazione $Z v_i = \sigma_i u_i$, e i σ_i sono le radici degli autovalori non nulli sia di $Z^\top Z$ sia di $Z Z^\top$. Quando $N \neq p$ si impiega la **SVD ridotta (economy)**, che trattiene solo le prime $r = \text{rank}(Z)$ colonne ed è più economica in calcolo e memoria.

PCA come caso particolare della SVD. Sostituendo la SVD nella matrice di correlazione,

$$R = \frac{1}{N-1} Z^\top Z = \frac{1}{N-1} V \Sigma^2 V^\top,$$

si vede che i **vettori singolari destri** V **coincidono con gli autovettori di** R — cioè con le direzioni principali (i *loadings*) — e che gli autovalori valgono $\lambda_i = \sigma_i^2 / (N-1)$. Gli **score** si ottengono direttamente come $PC = ZV = U\Sigma$. È questa la via seguita in pratica: anziché formare R e diagonalizzarla, si calcola la SVD di Z , numericamente più stabile (è ciò che fa `sklearn`).

Troncamento ottimo. Trattenendo le prime k colonne, $Z_k = U_k \Sigma_k V_k^\top$ è la **migliore approssimazione di rango** k di Z , con errore in norma di Frobenius $\|Z - Z_k\|_F^2 = \sum_{i > k} \sigma_i^2$. Massimizzare la varianza trattenuta e minimizzare l'errore di ricostruzione sono quindi lo **stesso problema**: ciò giustifica la scelta di k per soglia di varianza (§3.5).

3.3 Interpretazione Geometrica

La PCA effettua una **rotazione rigida** del sistema di riferimento, allineandolo agli assi dell'iperellissoide dei dati: PC1 individua la direzione di massima dispersione; PC2 è ortogonale a PC1 e massimizza la varianza residua; e così via. Si ottiene così una rappresentazione a dimensione ridotta che preserva la maggior parte della struttura.

3.4 Obiettivo nel dataset Student Performance

La PCA viene usata per

1. analizzare la struttura interna del dataset;
2. individuare gruppi di feature correlate tramite i *loadings*;
3. costruire rappresentazioni a dimensionalità ridotta da confrontare, in fase di classificazione, con lo spazio originale.

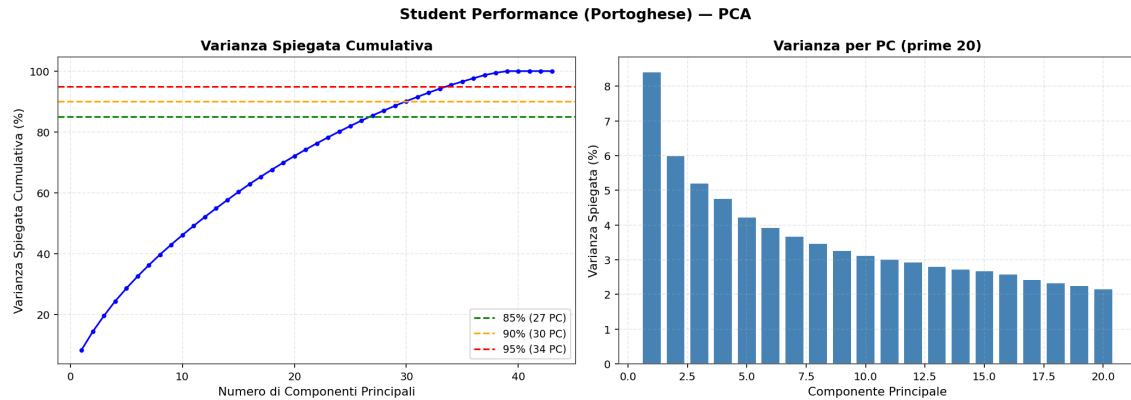


Figura 4: Varianza spiegata cumulativa (sinistra) e per singola componente (destra).

Tabella 2: Numero di componenti principali per soglia di varianza cumulativa (su 43 feature).

Soglia di varianza	85%	90%	95%
N. di componenti	27	30	34

3.5 Analisi della Varianza Spiegata (Scree Plot)

Calcolate le componenti, resta da capire quanta informazione ciascuna ne conservi. La traccia della matrice di correlazione, $\text{tr}(R) = \sum_i \lambda_i$, è la **varianza totale**, ripartita tra le componenti: la **proporzione di varianza spiegata** dal componente i -esimo (PVE_i) e quella **cumulativa** sulle prime k (PVE_{tot}) sono

$$\text{PVE}_i = \frac{\lambda_i}{\sum_{j=1}^p \lambda_j}, \quad \text{PVE}_{\text{tot}}(k) = \sum_{i=1}^k \text{PVE}_i = \frac{\sum_{i=1}^k \lambda_i}{\sum_{j=1}^p \lambda_j}.$$

Lo **Scree Plot** riassume visivamente queste quantità, riportando le PVE_i come barre e la PVE_{tot} come curva crescente.

Da questo grafico si decide quante componenti trattenere. Il modo più immediato è il **criterio del gomito (elbow)**: si cerca il punto in cui la curva degli autovalori smette di scendere bruscamente e si appiattisce, separando le poche componenti dominanti dalla coda di quelle che spiegano poco. In alternativa, e in modo più oggettivo, si fissa una **soglia di varianza cumulativa**, trattenendo il numero minimo di componenti necessario a superarla (qui il 95%). Come mostra il paragrafo seguente, su questo dataset il gomito non è individuabile, e sarà la soglia cumulativa a guidare la scelta.

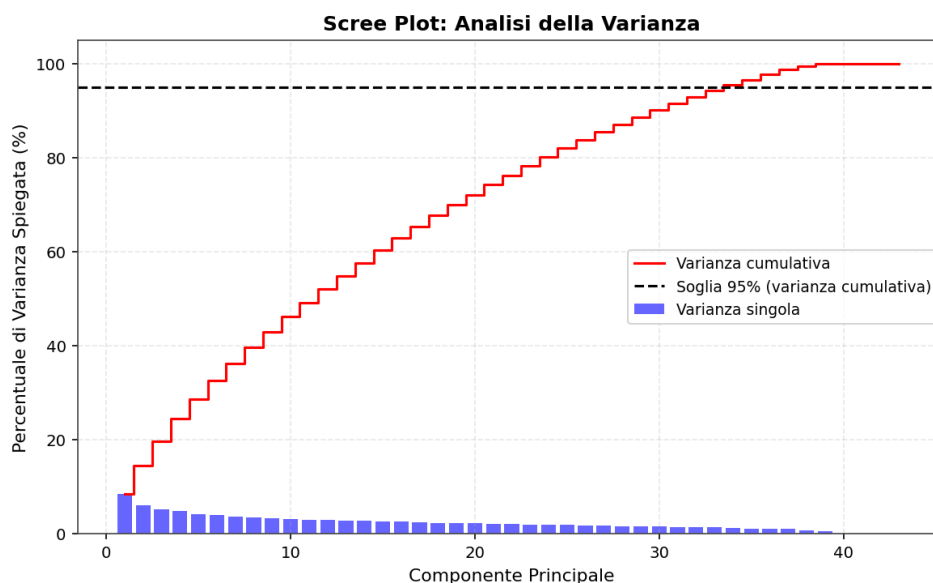


Figura 5: Scree plot: analisi della varianza.

3.6 Interpretazione dei Risultati: perché serve la soglia di varianza

La varianza non è distribuita uniformemente — le prime componenti contano più delle ultime — ma decade in modo **lento e continuo**: PC1 spiega appena l’8,4%, PC2 il 6,0%, PC3 il 5,2% (circa), e le successive calano di pochi decimi di punto alla volta, così che le prime tre insieme coprono solo ~20% del totale. Lo spettro degli autovalori è, in pratica, quasi **piatto**.

È proprio questa piattezza a rendere inservibile il criterio del gomito: non esistendo un gruppo di autovalori grandi nettamente staccato dalla coda (*spectral gap*), la curva non presenta alcun ginocchio e non offre quindi una soglia oggettiva. La ragione è **strutturale**. Dopo Label e One-Hot Encoding la matrice dei dati è dominata da variabili binarie e frammentate, correlate fra loro in modo debole e *diffuso*: non esiste un piccolo insieme di **fattori latenti** che generi i dati, e la (modesta) struttura di correlazione si distribuisce su molte direzioni quasi equivalenti. Uno spettro piatto significa esattamente questo — R è vicina a una matrice priva di direzioni privilegiate — e nessun sottospazio a bassa dimensione riesce a catturare la maggior parte dell’informazione.

In assenza di gomito, l’unico criterio dal significato operativo controllato è la soglia di varianza cumulativa. L’errore quadratico di ricostruzione troncando a k componenti vale $\|Z - Z_k\|_F^2 = \sum_{i>k} \lambda_i$, quindi imporre $\text{PVE}_{\text{tot}} \geq 95\%$ equivale a garantire una perdita relativa di informazione non superiore al 5%: un vincolo esplicito e riproducibile, che `scikit-learn` applica direttamente con `PCA(n_components=0.95)`. Quando gli autovalori sono tutti comparabili nessun troncamento è “gratuito” — scartare una componente costa quasi quanto scartarne un’altra — e fissare una soglia sull’informazione trattenuta è la decisione più difendibile.

Il risultato — **34 PC su 43** per il 95% (30 per il 90%, 27 per l’85%) — conferma che il dataset è **intrinsecamente ad alta dimensionalità**. Questo perché la complessità del comportamento degli studenti non può essere ridotta a pochi fattori banali, dato che il dataset è “denso” di informazioni distribuite su quasi tutte le 43 feature. La PCA offre dunque una compressione modesta: più che a ridurre drasticamente le dimensioni, qui serve a **decorrelare** le feature (poiché la PCA crea componenti che sono ortogonali tra loro) e a fornire una rappresentazione di confronto controllata per la fase di classificazione (§9).

Questa rappresentazione compressa X_{pca_95} verrà riusata più avanti (FDA/MDA su dati PCA, SVM su dati PCA).

3.7 Score Plot e Loading Graph

Poiché nessun piccolo gruppo di componenti domina, le prime PC non bastano a riassumere i dati, ma restano comunque utili per cercare pattern macroscopici. Aiutano in questo due strumenti complementari. Lo **Score Plot** proietta i campioni nello spazio 2D/3D delle nuove coordinate: colorando i punti per fascia di rendimento ($G3$) si può vedere se la PCA tende a mappare studenti eccellenti e insufficienti in regioni distinte. Il **Loading Graph** (*biplot*) proietta invece i versori della base originale nel nuovo spazio: l'orientamento e la lunghezza delle frecce assegnano un significato alle componenti — se ad esempio *absences* pesa molto lungo PC1, allora PC1 è legata all'assenteismo.

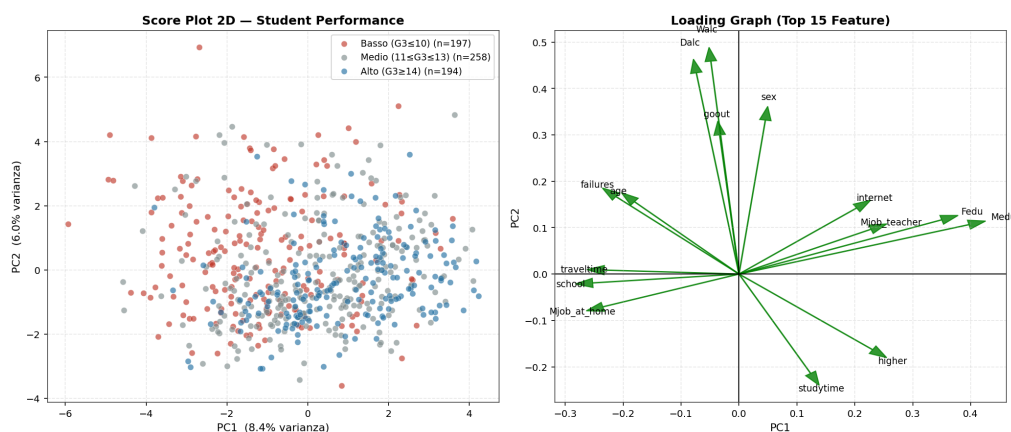


Figura 6: Score plot 2D (PC1-PC2) e loading graph.

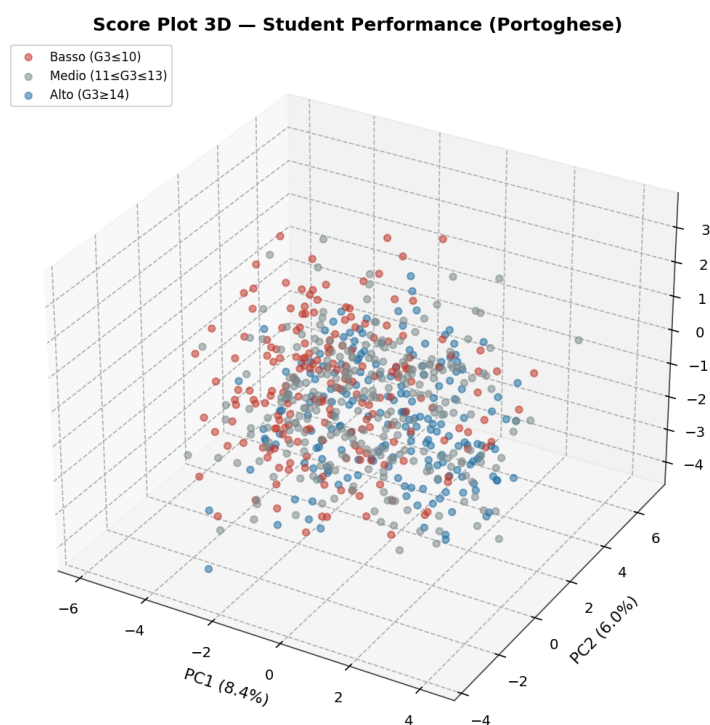


Figura 7: Score plot 3D (PC1-PC2-PC3) per fascia di rendimento.

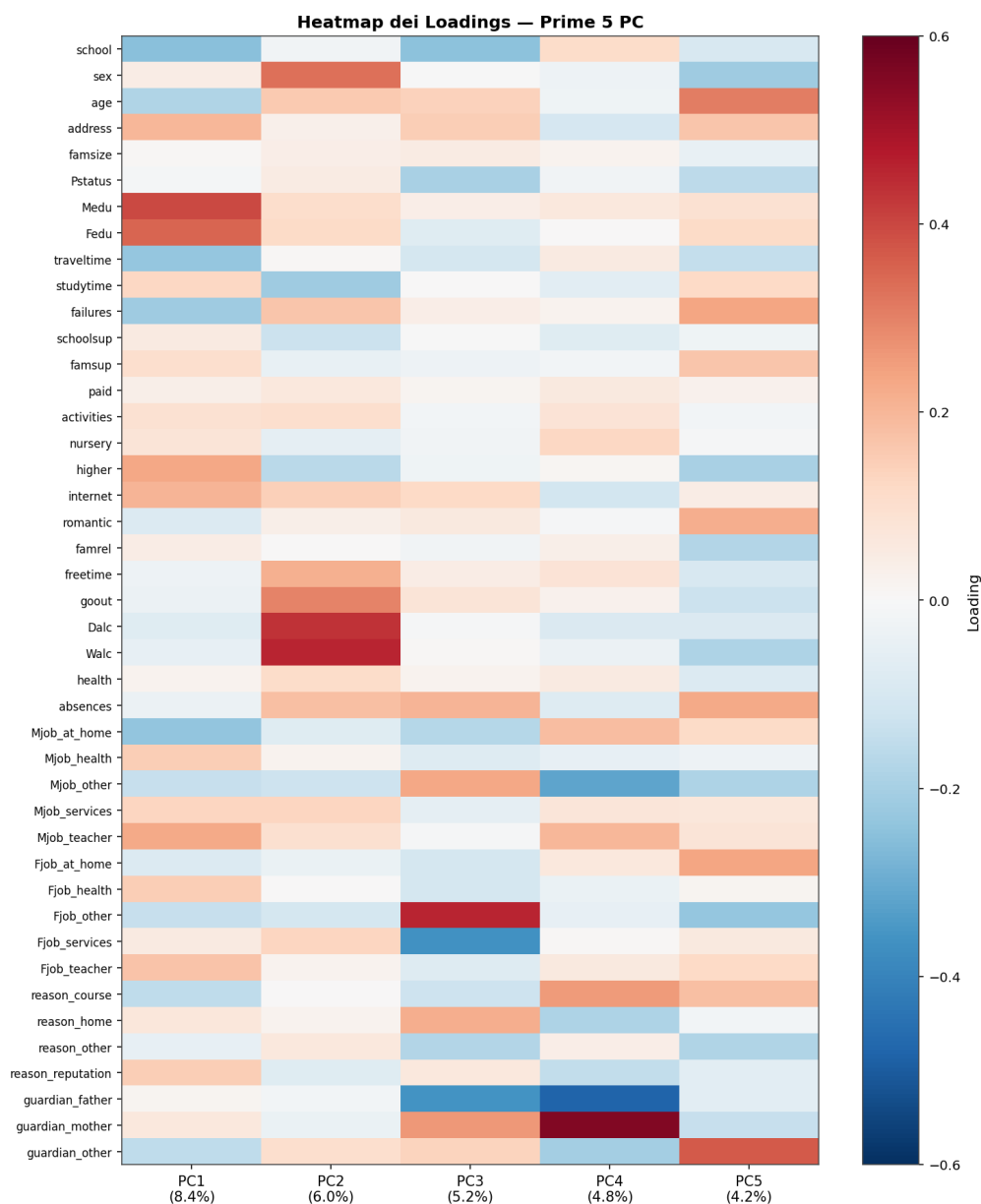


Figura 8: Heatmap dei loadings (contributo delle variabili originali) per le prime 5 componenti (28.6% di varianza cumulata; scala ± 0.6 , poco sopra il loading massimo in modulo $|L|_{\max} = 0.55$).

3.8 Osservazioni sui Risultati della PCA

Le osservazioni sperimentali confermano il quadro teorico. Sul piano della varianza la conferma è netta: nessun fattore latente dominante emerge e l'informazione resta spalmata su molte componenti, conseguenza diretta della natura eterogenea del dataset, in cui convivono variabili socio-demografiche, comportamentali e scolastiche poco correlate tra loro. Va inoltre osservato che la standardizzazione delle numerose variabili binarie e one-hot accentua di per sé l'appiattimento dello spettro — ogni *dummy* introduce una direzione quasi indipendente — per cui una parte della “piattezza” è un **effetto della codifica**, non solo una proprietà intrinseca del fenomeno.

Lo score plot mostra una **lieve separazione** tra studenti eccellenti (blu) e insufficienti (rosso), soprattutto lungo PC1, ma con forte sovrapposizione: le prime componenti catturano anche variabilità demografica e di stile di vita, non solo scolastica, e la fascia intermedia “Buono”

(grigio) occupa la regione centrale, risultando la più difficile da isolare.

L'analisi dei *loadings* permette infine di dare un significato semantico agli assi. PC1 è dominata da variabili di *impegno e capitale culturale* — aspirazione all'istruzione superiore (*higher*), istruzione dei genitori (*Medu*, *Fedu*) e bocciature pregresse (*failures*, in direzione opposta) — e può essere letta come asse **Impegno/Storico Accademico**. PC2 e PC3 raccolgono invece variabilità legata allo *stile di vita* — uscite (*goout*), consumo di alcol (*Dalc*, *Walc*), tempo libero (*freetime*) — configurandosi come asse **Stile di Vita/Socialità**. Va ricordato che i segni dei *loadings* sono definiti a meno di un cambio di segno complessivo, e possono quindi variare tra esecuzioni senza che l'interpretazione cambi.

In sintesi, la PCA riassume le numerose feature in pochi macro-assi comportamentali (Impegno *vs* Stile di Vita), confermando che il rendimento dipende da dinamiche socio-comportamentali complesse, non riassumibili in un'unica variabile lineare.

3.9 Score Plot con Target Binario

Finora gli score plot sono stati colorati secondo le **tre** fasce di rendimento. È utile osservare la stessa proiezione PCA anche rispetto a una versione **binaria** del target — **Promosso** ($G3 \geq 10$, soglia ufficiale di sufficienza nel sistema portoghese 0–20) contro **Bocciato** ($G3 < 10$). Questa partizione a due classi verrà ripresa nella sezione sull'analisi discriminante per costruire la FDA a un solo asse, ed è il riferimento naturale per leggere quanto la prima componente principale catturi già il segnale “promozione”.

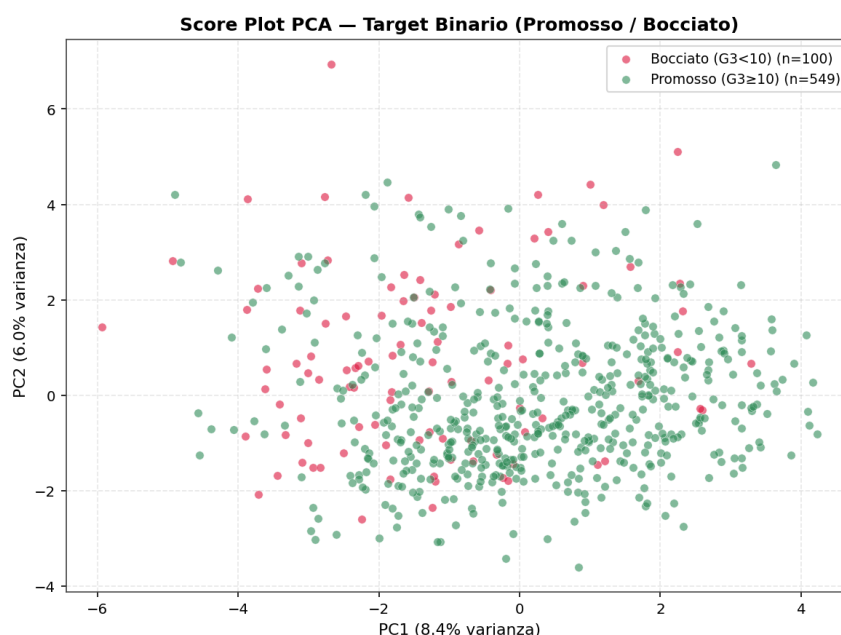


Figura 9: Score plot PCA per target binario (Promosso $G3 \geq 10$ / Bocciato $G3 < 10$).

4 Preparazione dei Dati per la Classificazione

Per i modelli supervisionati (MDA/LDA, SVM, K-NN, MLP) il target è la **fascia di rendimento** dello studente. Il voto finale $G3$ (scala 0–20) viene discretizzato in **3 fasce di numerosità confrontabile** ($\sim 30/40/30\%$), ottenute tagliando ai **quantili empirici** ($q_{33} \approx 11$, $q_{66} \approx 13$) di $G3$:

- **Basso** — $G3 \leq 10$ ($\sim 30\%$)
- **Medio** — $11 \leq G3 \leq 13$ ($\sim 40\%$)
- **Alto** — $G3 \geq 14$ ($\sim 30\%$)

Perché i terzili (giustificazione teorica del bilanciamento). I tagli $G3 = 11$ e $G3 = 14$ coincidono con i **quantili empirici** $q_{33} \approx 11$ e $q_{66} \approx 13$ di $G3$, così che le tre fasce abbiano numerosità simile (Tabella 3). La discretezza di $G3$ impedisce però fasce *esattamente* uguali: la fascia centrale resta la più popolata ($\sim 40\%$). Questa scelta è preferibile a soglie arbitrarie:

1. rende i **priori** delle classi *confrontabili* ($\approx 0.30/0.40/0.30$): la classe maggioritaria resta al $\sim 40\%$ (baseline $\approx 39.8\%$), quindi l'**accuratezza** non è del tutto attendibile da sola e va letta insieme alla **macro-F1** (§9.1), ma l'effetto della classe dominante è comunque molto ridotto rispetto a soglie arbitrarie;
2. la **LDA/QDA**, che modellano $\pi_k f_k(x)$, stimano confini di decisione più affidabili quando nessuna classe è marginale (la stima della covarianza per-classe della QDA degenera su classi piccole);
3. un target a **massima entropia** (H massima per distribuzione uniforme) trasporta la massima informazione e impedisce al classificatore di ottenere punteggi alti predicendo sempre la stessa classe;
4. il confronto con la **baseline** (classe maggioritaria) diventa più stringente e significativo.

Tabella 3: Distribuzione delle tre fasce (intero dataset, $N = 649$).

Fascia	N. studenti	%
Basso ($G3 \leq 10$)	197	30.4
Medio ($11 \leq G3 \leq 13$)	258	39.8
Alto ($G3 \geq 14$)	194	29.9

Il prezzo è che i tagli non coincidono con soglie “semantiche” (es. $G3 = 10$ = sufficienza): per questo la soglia di sufficienza è mantenuta separatamente nel **target binario** Promosso/Bocciato della §3.9, usato per la FDA a un asse.

Per valutare la **generalizzazione**, il dataset è suddiviso in **Training Set** (75%) e **Test Set** (25%) con campionamento **stratificato** (mantiene le proporzioni delle classi in entrambi i sottoinsiemi).

Nota metodologica (no data leakage). Diversamente dall’analisi descrittiva delle sezioni precedenti — dove la PCA/MDA esplorative operano sull’intero dataset standardizzato X_{scaled} — qui lo **split precede lo scaling**: si parte dai dati **grezzi** X e lo `StandardScaler` viene stimato (fit) **solo sul Training Set**, poi applicato (transform) sia al training sia al test. Così le statistiche di media e deviazione standard non incorporano alcuna informazione del test. Per coerenza, anche la PCA/MDA usate come pre-processing predittivo (§9) e la `GridSearchCV` di SVM ed MLP (tramite una Pipeline che standardizza dentro ogni fold sulla sola porzione di training) sono interamente *leakage-free*.

La **baseline** (classificatore banale che predice sempre la classe più numerosa, qui “Medio”) vale circa il **39.8%**: ogni modello va letto rispetto a questo riferimento.

5 Fisher / Multiple Discriminant Analysis (FDA / MDA)

5.1 Fondamenti Teorici

5.1.1 Idea e differenza con la PCA

La PCA cerca le direzioni di massima varianza **ignorando le etichette** (non supervisionata): ottime per descrivere i dati, ma nulla garantisce che separino le classi. L'**Analisi Discriminante di Fisher (FDA)** è invece **supervisionata**: cerca le direzioni lungo cui i **centroidi delle classi** sono il più possibile distanti tra loro, mentre i punti di una stessa classe restano addensati attorno al proprio centroide. In una parola, la PCA massimizza la varianza *totale*, la FDA massimizza la varianza *tra classi* relativa a quella *interna alle classi*.

5.1.2 Caso a due classi

Proiettando i dati su un versore v si ottengono scalari $\tilde{x} = v^\top x$. Detti $\tilde{\mu}_i = v^\top \mu_i$ i centroidi proiettati e $\tilde{S}_i^2 = \sum_{x_j \in C_i} (v^\top x_j - \tilde{\mu}_i)^2$ gli *scatter* proiettati, Fisher massimizza il rapporto

$$J(v) = \frac{(\tilde{\mu}_1 - \tilde{\mu}_2)^2}{\tilde{S}_1^2 + \tilde{S}_2^2}.$$

Il numeratore misura la **separazione tra le classi**, il denominatore la **dispersione interna**: si vuole il primo grande e il secondo piccolo. Introducendo le **matrici di scatter**

$$S_i = \sum_{x_j \in C_i} (x_j - \mu_i)(x_j - \mu_i)^\top, \quad S_W = S_1 + S_2, \quad S_B = (\mu_1 - \mu_2)(\mu_1 - \mu_2)^\top,$$

(*within-class* e *between-class*) e osservando che $\tilde{S}_i^2 = v^\top S_i v$ e $(\tilde{\mu}_1 - \tilde{\mu}_2)^2 = v^\top S_B v$, il discriminante diventa un **quoziente di Rayleigh**

$$J(v) = \frac{v^\top S_B v}{v^\top S_W v}.$$

Annullando il gradiente $\nabla_v J = 0$ si ottiene la condizione $S_B v = \lambda S_W v$, ossia il **problema agli autovalori generalizzato**

$$S_W^{-1} S_B v = \lambda v.$$

(se S_W ha rango pieno, cioè è definita positiva)

Nel caso a due classi, $S_B v$ è sempre parallelo a $(\mu_1 - \mu_2)$, da cui la soluzione esplicita $v^* \propto S_W^{-1}(\mu_1 - \mu_2)$: la direzione discriminante ottimale “raddrizza” la congiungente dei centroidi tenendo conto della forma (covarianza) delle nuvole.

5.1.3 Caso multiclasse (MDA)

Con C classi si generalizzano gli scatter pesando per la numerosità n_i :

$$S_B = \sum_{i=1}^C n_i (\mu_i - \mu)(\mu_i - \mu)^\top, \quad S_T = S_W + S_B,$$

dove μ è la media globale e S_T lo scatter totale (la decomposizione $S_T = S_W + S_B$ è l'analogo matriciale della scomposizione della varianza). Si massimizza ora $J(V) = \frac{\det(V^\top S_B V)}{\det(V^\top S_W V)}$, la cui soluzione sono i **primi autovettori** di $S_W^{-1} S_B$. Poiché S_B è somma di C matrici di rango 1 vincolate da $\sum_i n_i (\mu_i - \mu) = 0$, il suo **rango è al più $C - 1$** : esistono quindi **al più $C - 1$ assi discriminanti** non banali. Con $C = 3$ (qui) si ottengono **2 assi** — la *Multiple Discriminant Analysis* (MDA) — sufficienti per una visualizzazione 2D della separabilità. La rappresentazione ridotta di un dato è $y_i = V^\top x_i \in \mathbb{R}^{C-1}$.

5.1.4 Dalla proiezione al classificatore (LDA)

La FDA/MDA è una **riduzione di dimensionalità**; diventa un **classificatore** (LDA, §5.5.1) sotto un'ipotesi probabilistica: se le classi sono **gaussiane con la stessa matrice di covarianza** Σ , la regola di Bayes $\arg \max_k \pi_k f_k(x)$ produce funzioni discriminanti **lineari** $\delta_k(x) = x^\top \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^\top \Sigma^{-1} \mu_k + \ln \pi_k$, i cui confini $\delta_k = \delta_j$ sono **iperpiani**. Si dimostra che la base che diagonalizza $S_W^{-1} S_B$ coincide, a meno di un riscalamento, con lo spazio in cui questa regola opera: per questo la proiezione di Fisher e il classificatore `LinearDiscriminantAnalysis` di `scikit-learn` condividono gli stessi assi (spiegazione più accurata in §5.5.1). Rilassando l'ipotesi di covarianza comune (una Σ_k per classe) si ottiene la **QDA**, con confini quadratici (§5.5).

5.1.5 Interpretazione geometrica

La PCA ruota gli assi verso la massima dispersione *globale*; la MDA li ruota (e li scala secondo S_W) verso la massima *separabilità tra classi*. Per questo, a parità di dimensioni, una proiezione MDA tende a mostrare nuvole di classe più nette di una PCA — vantaggio che però dipende dalla validità delle ipotesi gaussiane e dalla reale separabilità dei dati.

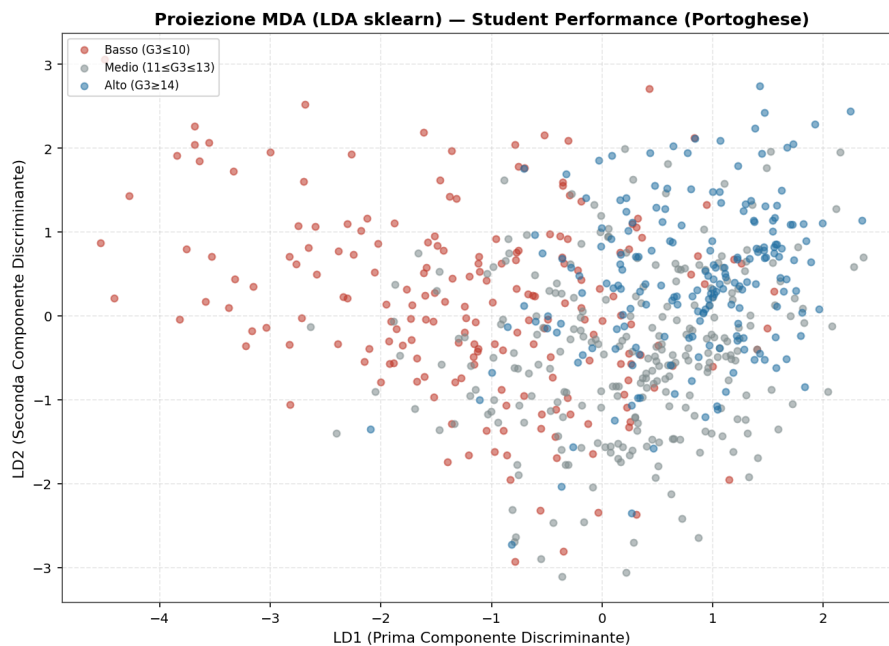


Figura 10: Proiezione MDA (LDA sklearn) sui 2 assi discriminanti.

5.2 Proiezione MDA con classe dedicata e confronto con la PCA

La proiezione discriminante viene ora calcolata con una classe dedicata, che risolve direttamente il problema agli autovalori generalizzato $S_B v = \lambda S_W v$ descritto sopra, estraendo i $C - 1$ assi di Fisher. Con $C = 3$ classi si ottengono **2 assi discriminanti**, che mettiamo a confronto, fianco a fianco, con lo score plot della **PCA** (non supervisionata) sullo stesso dataset. L'obiettivo è visualizzare la differenza concettuale: la PCA cerca le direzioni di massima dispersione *ignorando le etichette*, la MDA quelle di massima separabilità *tra classi note*.

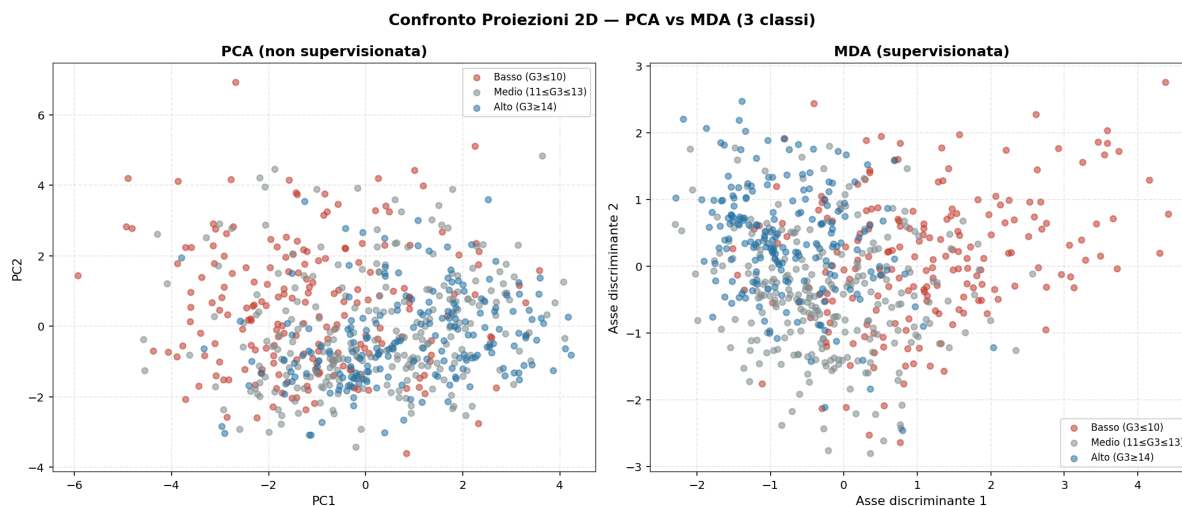


Figura 11: Confronto proiezioni 2D: PCA (non supervisionata) vs MDA (supervisionata).

Commento al confronto. I due pannelli rendono visibile la differenza tra i criteri. Nello score plot della **PCA** (sinistra) le tre fasce restano largamente sovrapposte: massimizzando la sola varianza totale, le prime due componenti catturano variabilità demografica e di stile di vita più che il segnale di classe. La proiezione **MDA** (destra), che massimizza la separabilità *tra* classi, allinea le nuvole lungo il primo asse discriminante e distanzia meglio le fasce Basso e Alto, lasciando la Medio interposta. Gli autovalori dei due assi ($\lambda_1 \approx 0.63$, $\lambda_2 \approx 0.14$) confermano che quasi tutto il potere discriminante è concentrato sul **primo asse**, mentre il secondo aggiunge poco (la maggior parte della separazione tra le fasce avviene lungo questa prima direzione). Resta comunque una sovrapposizione residua: le classi non sono linearmente separabili — un limite strutturale dei dati, non del metodo.

5.3 Caso Binario: FDA a un asse e confronto binario / multiclasse

Con due sole classi (Promosso / Bocciato) il metodo di Fisher fornisce **un solo asse discriminante** ($C - 1 = 1$): è la FDA in senso stretto. Confrontiamo la proiezione 1D della FDA con la prima componente principale (PC1) della PCA — entrambe ridotte a una dimensione e colorate per classe binaria — e affianchiamo il **caso binario** (FDA, 1 asse) al **caso multiclasse** (MDA, 2 assi), per evidenziare come il numero di assi disponibili dipenda dal numero di classi.

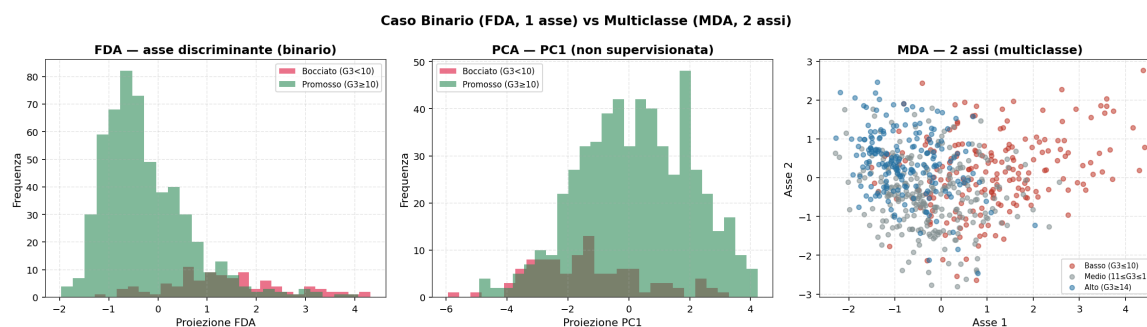


Figura 12: Caso binario (FDA, 1 asse) vs multiclasse (MDA, 2 assi).

Commento al confronto. La differenza supervisionato/non supervisionato è netta. L'asse **FDA** (sinistra) addensa Promossi e Bocciati attorno a due mode distinte, con sovrapposizione contenuta, mentre la **PC1** (centro) — costruita *ignorando* le etichette — mescola molto di più le due classi: la prima direzione di massima varianza non coincide con quella di massima separabilità. Il pannello **MDA** (destra) richiama il vincolo strutturale del metodo di Fisher: con C classi si hanno al più $C - 1$ assi, quindi il caso binario fornisce **una sola dimensione** (un istogramma) e quello a tre classi **due** (uno scatter). Il numero di assi disponibili dipende dal numero di classi, non dalla dimensione dei dati.

5.4 FDA e MDA sui dati compressi dalla PCA

Una pipeline frequente consiste nel **comprimere prima con la PCA** (riduzione non supervisionata del rumore) e applicare poi l'analisi discriminante di Fisher sullo spazio ridotto. Appliciamo quindi la FDA (binaria) e la MDA (multiclasse) alla rappresentazione X_pca_95 — le componenti che spiegano il 95% della varianza — e confrontiamo le proiezioni con quelle ottenute sullo spazio completo. Se il segnale discriminante sopravvive alla compressione, le due rappresentazioni risulteranno simili.

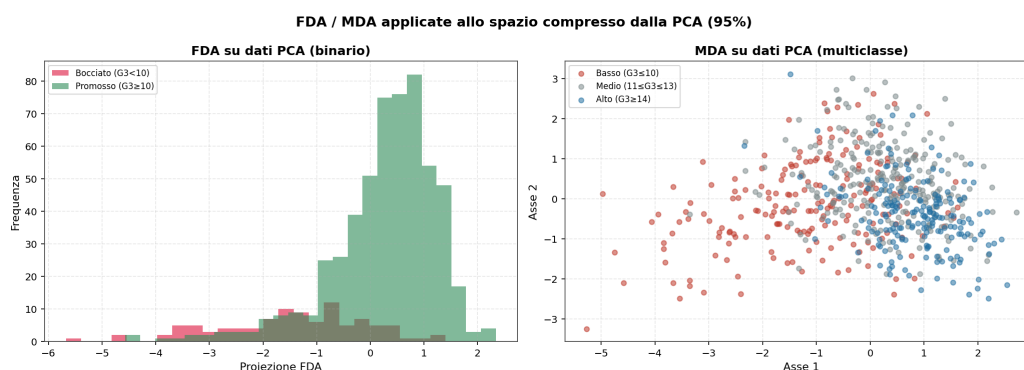


Figura 13: FDA/MDA applicate allo spazio compresso dalla PCA (95%).

Commento al confronto. Applicare FDA e MDA allo spazio già compresso dalla PCA al 95% (34 componenti) produce proiezioni praticamente sovrapponibili a quelle ottenute sulle 43 feature originali: la stessa separazione binaria nell'istogramma FDA e la stessa disposizione delle tre nuvole sugli assi MDA. Il segnale discriminante **sopravvive alla compressione**, perché la PCA scarta direzioni a bassa varianza che, in questo dataset, non portano informazione di classe. La riduzione non supervisionata può quindi servire da pre-processing senza distruggere ciò che è utile a Fisher, con il vantaggio di operare in uno spazio più piccolo — utile, ad esempio, a stabilizzare la stima della covarianza nella QDA (§5.5).

5.5 LDA e QDA come classificatori; confronto PCA / MDA / LDA

Va distinto l'uso **proiettivo** (FDA / MDA, riduzione di dimensionalità) da quello **classificativo**. La **Linear Discriminant Analysis (LDA)** assume classi gaussiane con **stessa matrice di covarianza** (omoschedasticità): ne derivano bordi di decisione **lineari**. La **Quadratic Discriminant Analysis (QDA)** rilassa questa ipotesi ammettendo una covarianza **diversa per ogni classe** (eteroschedasticità): i bordi diventano **quadratici**, a costo di stimare più parametri e di un rischio maggiore di overfitting su classi poco numerose. Qui **addestriamo e valutiamo la LDA come classificatore** (finora impiegata solo come proiezione in §5.1) e la confrontiamo con la **QDA**; infine, poiché sotto le ipotesi gaussiane la proiezione fornita dalla LDA coincide con quella della MDA a parte un riscalamento degli assi, mettiamo a confronto su tre pannelli le proiezioni 2D di **PCA**, **MDA** (classe custom) e **LDA** (scikit-learn).

5.5.1 Derivazione bayesiana di LDA e QDA

Entrambi i metodi modellano le classi come **gaussiane**, con densità condizionate

$$f_k(x) = \frac{1}{(2\pi)^{p/2} |\Sigma_k|^{1/2}} \exp\left(-\frac{1}{2}(x - \mu_k)^\top \Sigma_k^{-1} (x - \mu_k)\right),$$

e applicano la **formula di Bayes** $\arg \max_k \pi_k f_k(x)$, equivalente a massimizzare $\delta_k(x) = \ln \pi_k + \ln f_k(x)$. Con una covarianza **diversa per classe** si ottiene il discriminante **quadratico** della QDA,

$$\delta_k(x) = -\frac{1}{2} \ln |\Sigma_k| - \frac{1}{2} (x - \mu_k)^\top \Sigma_k^{-1} (x - \mu_k) + \ln \pi_k, \quad k = 1, \dots, c.$$

dove:

- $\pi_k \in (0, 1)$ è la **probabilità a priori** della classe k — la sua frequenza relativa nel training, $\pi_k = N_k / N$ con $\sum_k \pi_k = 1$ — e misura quanto la classe è comune *prima* di osservare x (con priori uniformi, e.g., $\pi_1 = \dots = \pi_c = 1/c$).
- $f_k(x) = p(x \mid y = k)$ è la **densità condizionata**: la verosimiglianza di osservare x se appartenesse alla classe k , qui assunta gaussiana di media μ_k e covarianza Σ_k (formula sopra).
- Σ_k è la **matrice di covarianza** della classe k , stimata campionariamente da S_k : ne descrive forma, dispersione e orientamento (varianze delle feature sulla diagonale, covarianze fuori diagonale). La differenza tra LDA e QDA sta tutta qui — la QDA ne stima una *per ogni classe*, la LDA una sola *comune*.

Infine $\delta_k(x)$ è la **funzione discriminante**, il punteggio assegnato a ciascuna classe (il logaritmo della posteriori a meno di costanti comuni): $\arg \max_k \pi_k f_k(x)$ equivale quindi a scegliere la classe con $\delta_k(x)$ massimo.

Assumendo invece una covarianza **comune** $\Sigma_k = \Sigma$, i termini quadratici in x si cancellano nel confronto $\delta_k - \delta_j$ e restano discriminanti **lineari** $\delta_k(x) = x^\top \Sigma^{-1} \mu_k - \frac{1}{2} \mu_k^\top \Sigma^{-1} \mu_k + \ln \pi_k$ (LDA), con confini iperpiani. La QDA stima $O(p^2)$ parametri per classe: in 43 dimensioni e con classi di poche centinaia di campioni le sue stime di covarianza degenerano (overfitting) — un'altra faccia della maledizione della dimensionalità (§2.1), che ne spiega in anticipo il crollo osservato sotto.

5.5.2 LDA come classificatore (baseline)

Addestro ora la LDA come **classificatore** sul training set e la valutiamo sul test set: è la baseline con cui confrontare la QDA e, più avanti, gli altri modelli. Fit e predizione sono tenuti distinti dalla proiezione di §5.1, che aveva scopo solo visivo.

Tabella 4: Report di classificazione — LDA (accuratezza **55.83%**).

	Basso	Medio	Alto	Macro	Pesata
Precision	0.56	0.53	0.61	0.57	0.56
Recall	0.49	0.65	0.51	0.55	0.56
F1	0.52	0.58	0.56	0.55	0.56

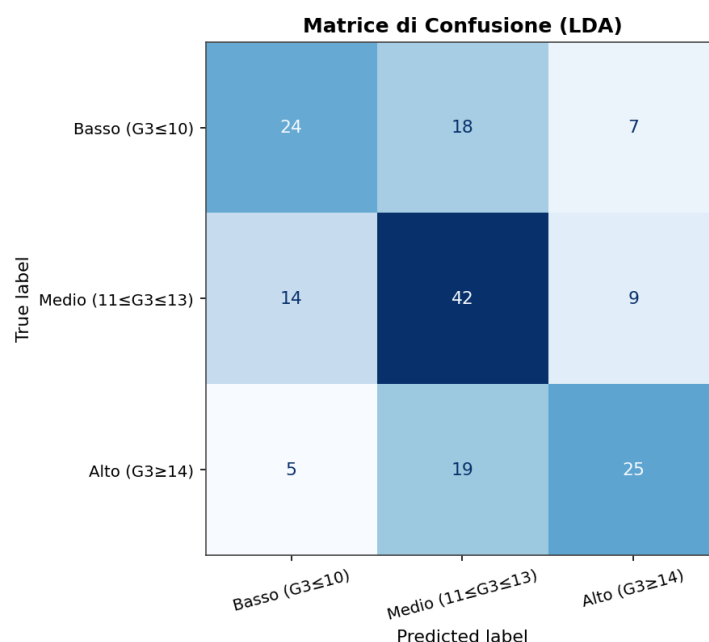


Figura 14: Matrice di confusione — LDA.

Ripeto la stessa valutazione con la **QDA**:

Tabella 5: Report di classificazione — QDA (accuratezza **40.49%**).

	Basso	Medio	Alto	Macro	Pesata
Precision	0.54	0.37	0.38	0.43	0.42
Recall	0.29	0.11	0.92	0.44	0.40
F1	0.37	0.17	0.54	0.36	0.34

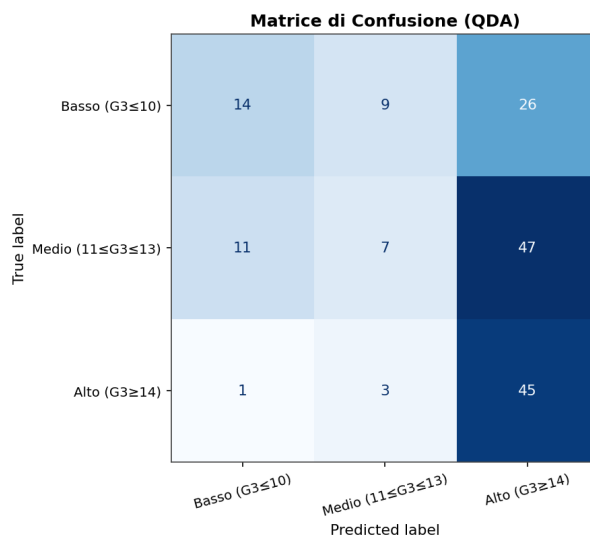


Figura 15: Matrice di confusione — QDA.

5.5.3 QDA sui dati compressi dalla PCA

La QDA stima una **covarianza piena per classe**: su 43 feature richiede circa $43 \cdot 44/2 \approx 950$ parametri per classe, con solo 160 campioni per classe nel training. È esattamente la situazione sotto-determinata descritta in §2.1. Ripetiamo quindi la QDA sullo spazio ridotto dalla **PCA al 95%**, stimata **solo sul training set**, per verificare se la riduzione stabilizza la stima.

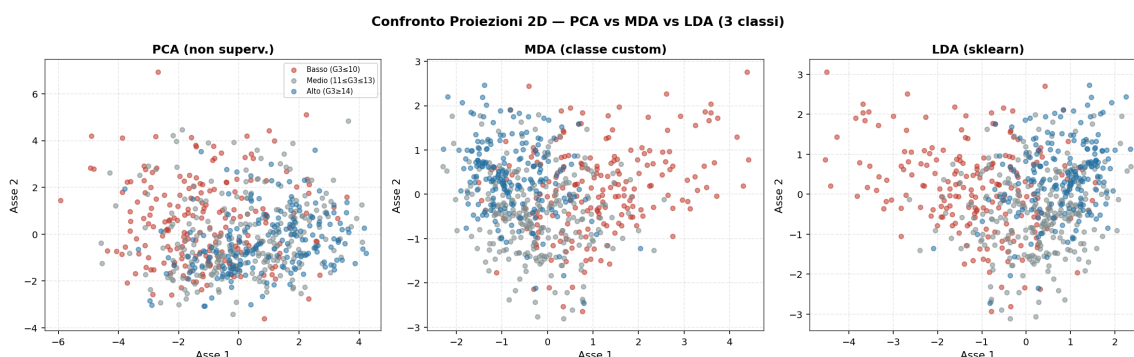


Figura 16: Confronto proiezioni 2D: PCA vs MDA vs LDA (3 classi).

Commento al confronto. Sul piano *classificativo* la **LDA** raggiunge il **55.83%** sul test, mentre la **QDA** crolla al **40.49%** — sotto la LDA e di poco sopra la baseline ($\sim 39.8\%$). È quanto anticipato in §2.1 e §5.5.1: la QDA stima una covarianza piena per classe (≈ 950 parametri su 43 feature) con poche centinaia di campioni per fascia, così le stime degenerano e il modello va in **overfitting**; vi contribuisce anche la **ridondanza** introdotta dalla codifica delle variabili categoriche (feature fortemente correlate, cfr. §2), che rende le covarianze per classe ancora meno affidabili (recall quasi tutto concentrato sulla classe “Alto”). Ripetendo la QDA sullo spazio **compressso dalla PCA al 95%** (34 componenti, stimata solo sul training) l’accuratezza risale al **53.37%**: ridurre p abbatte il numero di parametri e stabilizza la stima, rendendo la QDA di nuovo competitiva con la LDA. Sul piano *proiettivo*, i tre pannelli mostrano che **MDA** (classe custom) e **LDA** (scikit-learn) danno proiezioni quasi identiche — a meno di un riscaldamento/riflessione degli assi, come atteso sotto le ipotesi gaussiane — mentre la **PCA** mescola le classi: conferma visiva che, per separare, è necessario usare le etichette.

6 Support Vector Machine (SVM)

6.1 Fondamenti Teorici

Le **Support Vector Machine (SVM)** cercano l'**iperpiano** $w^\top x + b = 0$ che separa le classi **massimizzando il margine**, cioè la distanza tra l'iperpiano e i campioni più vicini (i *vettori di supporto*). Nel caso separabile il problema primale è

$$\begin{aligned} \min_{w,b} \quad & \frac{1}{2} \|w\|^2 \\ \text{s.t.} \quad & y_i (w^\top x_i + b) \geq 1. \end{aligned}$$

Per gestire classi non perfettamente separabili si introducono le **variabili di slack** $\xi_i \geq 0$ e il parametro di penalizzazione C (*soft margin*, iperparametro di regolarizzazione):

$$\begin{aligned} \min_{w,b,\xi} \quad & \frac{1}{2} \|w\|^2 + C \sum_i \xi_i \\ \text{s.t.} \quad & y_i (w^\top x_i + b) \geq 1 - \xi_i. \end{aligned}$$

C regola il *trade-off* tra ampiezza del margine e numero di errori: C grande penalizza fortemente gli errori (margine stretto, rischio overfitting), C piccolo favorisce un margine ampio. Nella formulazione **duale** i dati compaiono solo tramite prodotti scalari $x_i^\top x_j$, che possono essere sostituiti da una **funzione kernel** $K(x_i, x_j)$ (che rappresenta la *somiglianza* tra i due punti): ciò consente confini **non lineari** proiettando implicitamente i dati in uno spazio a dimensione superiore. Usiamo il kernel **RBF (gaussiano)**

$$K(x_i, x_j) = \exp\left(-\frac{\|x_i - x_j\|^2}{2\sigma^2}\right),$$

dove σ è un parametro di tuning (regolazione): se è piccolo, la decrescita del kernel è veloce, altrimenti la decrescita è lenta. Per semplicità di calcoli denoto

$$\frac{1}{2\sigma^2} = \gamma,$$

Ottimizzo C , γ e il tipo di kernel tramite **Grid Search** con **5-fold cross-validation**.

6.1.1 Formulazione duale e vettori di supporto

L'iperpiano ottimo realizza un'**ampiezza del margine** (banda tra i due iperpiani $w^\top x + b = \pm 1$) pari a $2/\|w\|$ — equivalentemente, un **margine geometrico** (distanza dall'iperpiano al punto più vicino) di $1/\|w\|$ — da cui l'equivalenza tra massimizzare il margine e minimizzare $\|w\|^2$. Associando ai vincoli i moltiplicatori di Lagrange $\alpha_i \geq 0$ e annullando le derivate rispetto a w, b, ξ si ottiene $w = \sum_i \alpha_i y_i x_i$ e il **problema duale**

$$\begin{aligned} \max_{\alpha} \quad & \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y_i y_j x_i^\top x_j \\ \text{s.t.} \quad & 0 \leq \alpha_i \leq C, \quad \sum_i \alpha_i y_i = 0. \end{aligned}$$

Per le condizioni di **KKT** solo i punti con $\alpha_i > 0$ contribuiscono a w : sono i **vettori di supporto** (ben classificati e fuori dal margine $\Rightarrow \alpha_i = 0$; sul margine $\Rightarrow 0 < \alpha_i < C$; dentro il margine o mal classificati $\Rightarrow \alpha_i = C$). Un margine più largo (C piccolo) ammette più vettori di supporto e generalizza meglio (§6.3). Poiché nel duale i dati compaiono **solo via prodotti scalari** $x_i^\top x_j$, questi si sostituiscono con una **funzione kernel** $K(x_i, x_j)$ (*kernel trick*) e la decisione diventa $f(x) = \text{sign}(\sum_i \alpha_i y_i K(x_i, x) + b)$, con $b = y_i - \sum_j \alpha_j y_j K(x_i, x_j)$

La configurazione ottima trovata dalla Grid Search è kernel RBF con $C = 1$ e $\gamma \approx 0.0155$ ($= 1/(1.5 \cdot 43)$): accuratezza media in cross-validation del **57.61%** e **54.60%** sul test set (Tabella 6).

Tabella 6: Report di classificazione — SVM RBF ($C=1$, $\gamma \approx 0.0155$; accuratezza **54.60%**).

	Basso	Medio	Alto	Macro	Pesata
Precision	0.58	0.55	0.51	0.55	0.55
Recall	0.57	0.48	0.61	0.55	0.55
F1	0.58	0.51	0.56	0.55	0.54

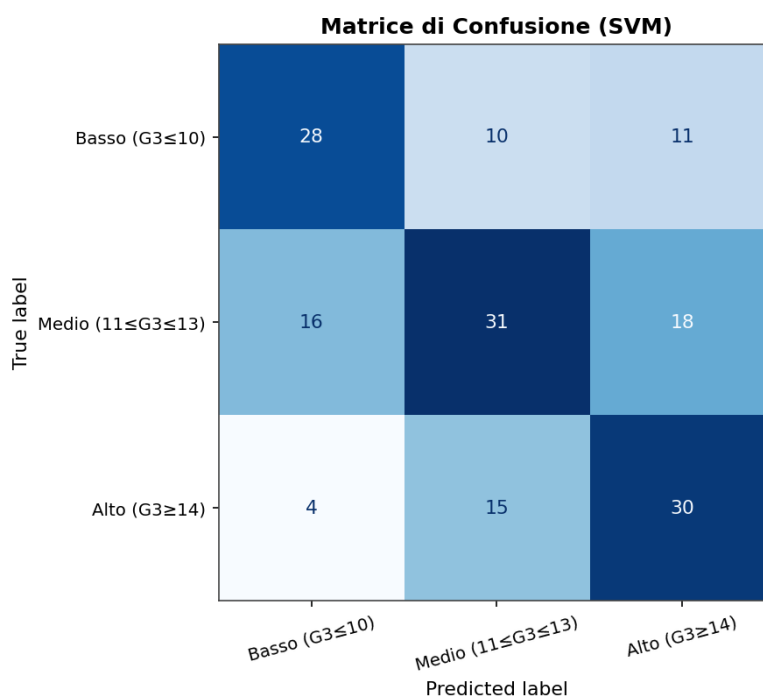


Figura 17: Matrice di confusione — SVM.

6.2 Margine Rigido vs Margine Morbido (Hard vs Soft Margin)

La SVM a **margine morbido** (parametro C) tollera alcune violazioni del margine tramite le variabili di slack ξ_i ; la SVM a **margine rigido** richiede invece la separazione perfetta, senza alcuna violazione. scikit-learn non offre una classe dedicata al margine rigido: lo si **approssima con un C molto grande**, che penalizza quasi all'infinito ogni violazione. Per questo confronto lineare usiamo **LinearSVC**, la classe dedicata alle SVM lineari: risolve il primale (`loss='squared_hinge'`, `dual=False`) tramite liblinear. A differenza della SVC con kernel usata nel resto del §6, LinearSVC non identifica i singoli vettori di supporto (liblinear non li espone come libsvm): il proxy di complessità diventa quindi l'**ampiezza del margine** $2/\|w\|$ derivata in §6.2. Confrontiamo

una SVM a margine morbido (C piccolo, margine ampio) con una a margine (quasi) rigido (C enorme, margine stretto), riportando ampiezza del margine e accuratezza su training e test: tipicamente il margine rigido **overfitta** il training set.

Tabella 7: SVM lineare (LinearSVC): margine morbido vs (quasi) rigido.

	Margine $2/\ w\ $	Acc. train	Acc. test
Margine morbido ($C=0.05$)	3.882	65.6%	55.8%
Margine rigido ($C=10^4$)	3.503	65.4%	55.8%

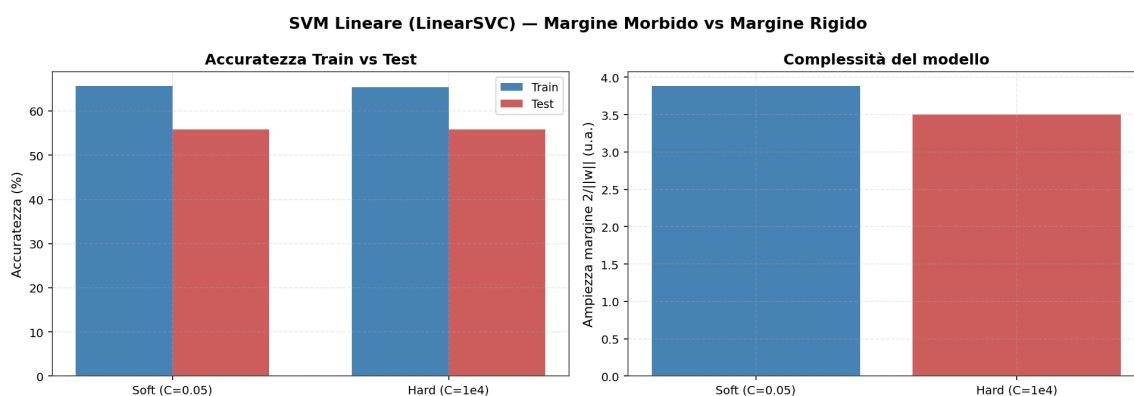


Figura 18: SVM lineare: accuratezza train/test e ampiezza del margine.

6.3 Soft SVM sui dati compressi dalla PCA

Applichiamo la SVM con gli **iperparametri ottimi** trovati dalla Grid Search alla rappresentazione compressa dalla PCA (95% della varianza). Il confronto con lo spazio originale verifica se la riduzione *non supervisionata* preserva l'informazione discriminante sfruttata dalla SVM.

Sullo spazio originale la SVM ottiene **54.60%**, mentre sullo spazio PCA al 95% (34 componenti) scende a **52.15%**: la riduzione non supervisionata comprime anche un po' di segnale utile.

6.4 Visualizzazione del confine di decisione (proiezione LDA 2D)

Disegno il **piano di separazione** appreso dalla SVM insieme ai **punti**. Poiché lo spazio originale ha molte feature, proietto su 2 dimensioni; uso la **LDA** (2 componenti, il massimo con 3 classi) invece della PCA perché la LDA *sfrutta le etichette* e massimizza la separazione tra le fasce (cfr. §5), dando una vista molto più leggibile del confine. La proiezione è stimata **solo sul training** (nessun leakage) e vi si addestra una SVM con gli stessi iperparametri ottimi: è una vista *illustrativa* in 2D, non il modello ufficiale a piena dimensionalità.

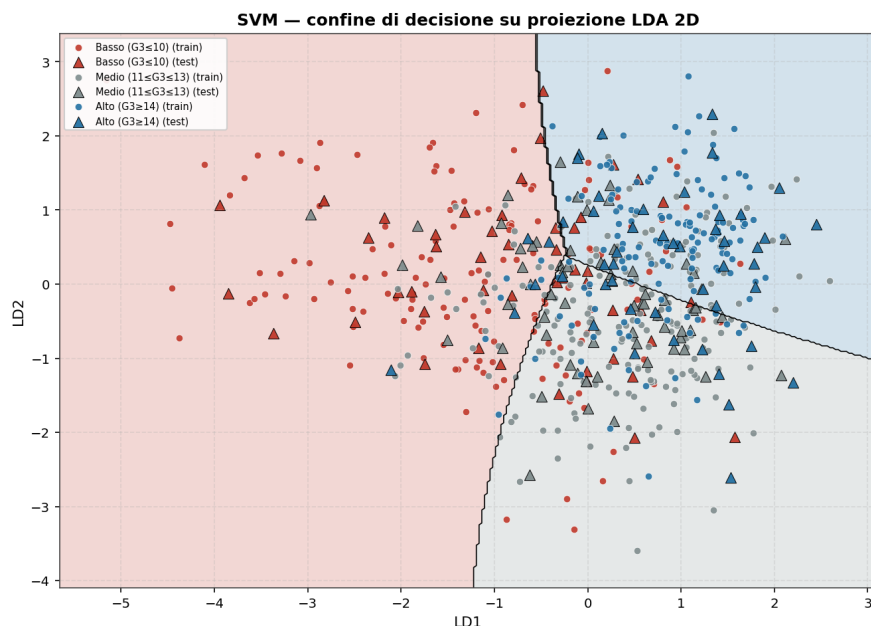


Figura 19: SVM — confine di decisione su proiezione LDA 2D (training: cerchi; test: triangoli).

6.5 Nota sulle SVM non lineari e multiclasse

Il kernel **RBF** impiegato nella Grid Search realizza già confini **non lineari** (cfr. §6.1). Con più di due classi, l'implementazione `SVC` di `scikit-learn` ricorre, per $m > 2$ classi, allo schema **One-vs-One**: addestra una SVM binaria per ogni coppia di classi e combina i voti; il metodo `decision_function` restituisce punteggi OvO, non probabilità. Questo spiega la flessibilità dei bordi attorno alla nuvola centrale della fascia “Buono”, la più difficile da isolare.

7 K-Nearest Neighbors (K-NN)

7.1 Fondamenti Teorici

Il **K-NN** è un metodo *lazy* basato sulle distanze (tipicamente euclidea). Per classificare un nuovo studente ne cerca i **K vicini più prossimi** in distanza euclidea e gli assegna la classe più votata, pesando il voto con l'inverso della distanza (i vicini più vicini contano di più — utile con 3 classi per evitare pareggi).

La scelta di K governa il **bias-variance trade-off**:

- K **piccolo** (es. $K = 1$): confini molto frastagliati che inseguono il rumore → bassa distorsione ma alta varianza (*overfitting*);
- K **grande**: confini lisci e stabili → bassa varianza ma alta distorsione (*underfitting*), con tendenza a predire la classe più numerosa.

Il K-NN soffre inoltre la **maledizione della dimensionalità** (§2.1): in spazi ad alta dimensione (qui 43 feature) le distanze euclidee si concentrano e perdono potere discriminante — il vicino “più prossimo” diventa poco informativo (di qui il netto recupero sulla proiezione MDA a 2D, §9). Esploriamo K da 1 a 39 confrontando accuratezza su train e test.

Selezione di K con un Validation Set. Scegliere K guardando direttamente il *test set* introdurre

rebbe una forma di *leakage*: il test verrebbe usato per impostare il modello. Si ritaglia quindi dal training un **validation set** (20%) su cui valutare ogni K ; il K migliore viene poi **ri-addestrato sull'intero training set** e valutato **una sola volta** sul test set. Si rispetta così la separazione Train / Validation / Test.

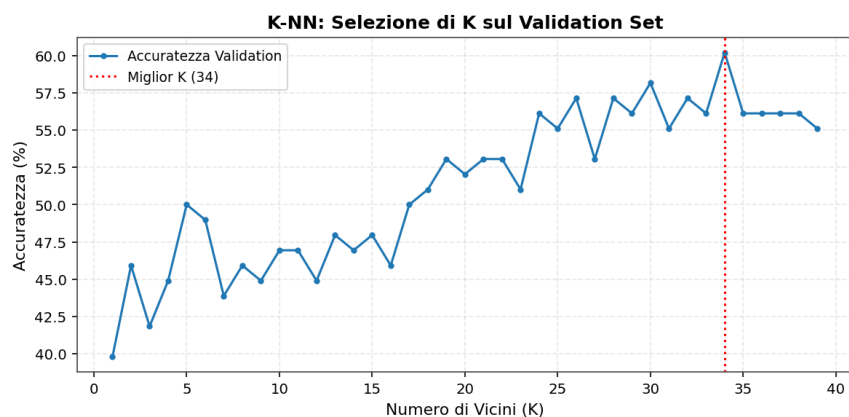


Figura 20: Selezione di K sul validation set.

Il valore selezionato è $K = 34$, con cui il K-NN ri-addestrato sull'intero training raggiunge **53.37%** sul test set (Tabella 8).

Tabella 8: Report di classificazione — K-NN ($K=34$; accuratezza 53.37%).

	Basso	Medio	Alto	Macro	Pesata
Precision	0.68	0.50	0.52	0.57	0.56
Recall	0.35	0.66	0.55	0.52	0.53
F1	0.46	0.57	0.53	0.52	0.53

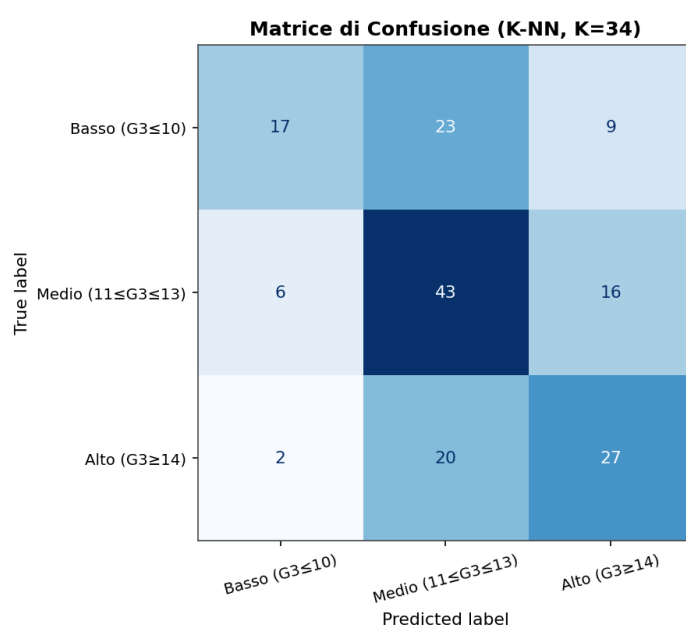


Figura 21: Matrice di confusione — K-NN ($K=34$).

7.2 K-NN sui dati compressi dalla PCA

Il K-NN è il classificatore più esposto alla maledizione della dimensionalità (§2.1): in alta dimensione le distanze euclidee si concentrano e il “vicino più prossimo” perde potere discriminante. Verifichiamo l’effetto riducendo lo spazio con la **PCA al 95%**, stimata **solo sul training set** (nessun leakage). Per coerenza con la tabella di confronto (§9) si riusa lo stesso K^* selezionato in §7.1.

Sullo spazio originale (43 feature) il K-NN ottiene **53.37%**, mentre sullo spazio PCA al 95% scende a **49.08%**: qui la PCA non restituisce potere discriminante alla metrica.

8 Multi-Layer Perceptron (MLP)

8.1 Fondamenti Teorici

Il **Multi-Layer Perceptron (MLP)** è una **rete neurale feed-forward** con $L + 1$ livelli, indicizzati da $l = 0, \dots, L$: il livello di input ($l = 0$) codifica il vettore delle feature $\mathbf{a}_0 = \mathbf{x}$, i livelli intermedi ($l = 1, \dots, L - 1$) sono gli *hidden layer* e il livello di output ($l = L$) produce la predizione $g(\mathbf{x})$. Il numero di livelli senza l’input, L , è la **profondità (depth)** della rete, mentre $\max_l p_l$ (numero massimo di nodi in un livello) ne è la **larghezza (width)**.

Ogni livello trasforma il precedente combinando una parte **lineare** e una **non lineare**:

$$\mathbf{z}_l = W_l \mathbf{a}_{l-1} + \mathbf{b}_l, \quad \mathbf{a}_l = S_l(\mathbf{z}_l),$$

dove W_l è la matrice dei pesi, $\mathbf{b}_l$ il vettore dei *bias* e S_l una **funzione di attivazione** applicata componente per componente. Nei livelli nascosti l’attivazione S è comune a tutti i nodi; in output, per la classificazione multi-classe, S_L è la **softmax** $\text{softmax}(\mathbf{z}) = e^{\mathbf{z}} / \sum_k e^{z_k}$ e la classe predetta è $\arg \max_k g_k(\mathbf{x})$ (classificatore *multi-logit*). L’intera rete è quindi la **composizione di funzioni**

$$g(\mathbf{x}) = S_L \circ M_L \circ \dots \circ S_1 \circ M_1(\mathbf{x}), \quad M_l(\mathbf{z}) = W_l \mathbf{z} + \mathbf{b}_l,$$

calcolata in avanti dall’algoritmo di **feed-forward propagation**. Grazie alla non linearità delle attivazioni l’MLP è un **approssimatore universale** (già una rete con un solo hidden layer può rappresentare qualunque funzione continua, cfr. teorema di Kolmogorov): aumentando profondità e larghezza cresce la **capacità rappresentativa** della classe di funzioni. Vale però un **trade-off approssimazione–stima**: la classe deve essere abbastanza ricca da rappresentare la funzione ottima, ma abbastanza semplice da poter essere addestrata con basso errore di stima e costo contenuto. Su un dataset piccolo come il nostro questo si traduce nel rischio di **overfitting**, che controlliamo con la regolarizzazione L_2 (parametro α) e l’**early stopping** (early_stopping=True: arresto quando il *punteggio di validazione* — l’accuratezza calcolata su una frazione validation_fraction=0.1 del training — non migliora per un certo numero di epoche); la *loss curve* di training ne documenta la convergenza.

8.1.1 Scelta della funzione di attivazione: perché ReLU

Il criterio guida è usare le attivazioni più semplici che permettano di costruire un learner con grande capacità rappresentativa e basso costo di addestramento. Confronto tre attivazioni possibili:

- **Heaviside** (gradino, $\mathbb{1}\{z > 0\}$): ha derivata nulla quasi ovunque, quindi è **inutilizzabile** con l'addestramento a gradiente (back-propagation).
- **Logistica / sigmoide** ($(1 + e^{-z})^{-1}$): derivabile, ma **satura** per $|z|$ grande ($S'(z) \rightarrow 0$); nella ricorsione della back-propagation ($\delta_{l-1} = D_{l-1} W_l^\top \delta_l$, con $D_l = \text{diag}(S'(z_l))$) ciò provoca **gradienti evanescenti** e rallenta la convergenza nelle reti profonde.
- **ReLU** ($\max(0, z) = z \cdot \mathbb{1}\{z > 0\}$): è la funzione più semplice e a **costo computazionale minimo** (la derivata vale 0 o 1, senza esponenziali); mantenendo derivata unitaria per $z > 0$ **non satura sul lato positivo**, così il gradiente si propaga bene attraverso i livelli. Resta non lineare — quindi preserva la capacità rappresentativa — evitando i problemi di gradiente della sigmoide.

Per questi motivi scelgo $S = \text{ReLU}$ nei livelli nascosti: è il miglior compromesso tra semplicità/costo e stabilità dell'addestramento. La **softmax** resta invece in output perché è l'attivazione naturale del classificatore multi-logit.

8.1.2 Funzione di costo, back-propagation e ottimizzazione

Raccolgo tutti i parametri in $\theta = \{W_l, b_l\}_{l=1}^L$. L'addestramento minimizza la **loss di training** media

$$\ell_\tau(\theta) = \frac{1}{n} \sum_{i=1}^n C_i(\theta), \quad C_i(\theta) = \text{Loss}(y_i, g(x_i | \theta)).$$

Con output softmax la Loss naturale è l'**entropia incrociata** $C = -\sum_k t_k \ln \hat{y}_k$ (con t etichetta one-hot), che dà un errore di output particolarmente semplice, $\delta_L = \hat{y} - t$.

I gradienti si ottengono in modo efficiente con la **back-propagation**. Posto $\delta_l := \partial C / \partial z_l$ e $D_l = \text{diag}(S'(z_l))$, il teorema del gradiente fornisce

$$\frac{\partial C}{\partial W_l} = \delta_l a_{l-1}^\top, \quad \frac{\partial C}{\partial b_l} = \delta_l, \quad \delta_{l-1} = D_{l-1} W_l^\top \delta_l,$$

cioè l'errore viene propagato **all'indietro** dal livello di output verso l'input tramite la regola della catena. Il costo è essenzialmente quello di alcune moltiplicazioni matriciali.

Noti i gradienti, i parametri si aggiornano per **discesa del gradiente** $\theta_{t+1} = \theta_t - \alpha_t u_t$, con $u_t = \partial \ell_\tau / \partial \theta$ e α_t *learning rate*. Nel MLPClassifier implementato uso `solver='adam'`, un metodo del primo ordine di tipo stocastico a mini-batch (variante adattiva della Stochastic Gradient Descent o della Steepest Descent), adatto a dataset di dimensione medio-piccola.

8.2 Selezione degli iperparametri (Grid Search + Cross-Validation)

Come per l'SVM (Sez. 6.1), anziché fissare a mano l'architettura e la regolarizzazione, seleziono `hidden_layer_sizes` e `alpha` per **5-fold cross-validation** su una Pipeline (StandardScaler + MLP), così la scelta è **empirica** e priva di *data leakage* (ogni fold ri-standardizza solo la propria porzione di training). Il numero massimo di epoche resta **fuori** dalla griglia: è solo un limite superiore, perché l'arresto effettivo è deciso dall'**early stopping**.

Gli iperparametri ottimi risultano `hidden_layer_sizes=(150,50)` e `alpha=0.1`; l'MLP raggiunge **53.99%** sul test set, arrestandosi a 31 epoche per early stopping (Tabella 9).

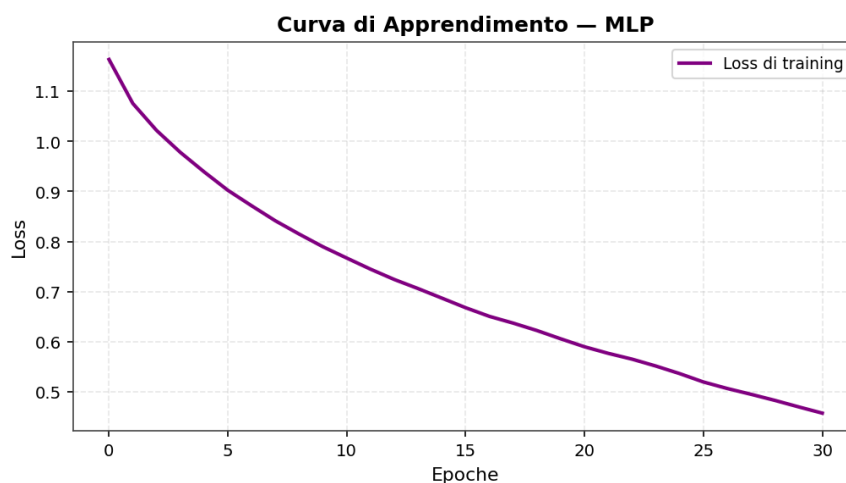


Figura 22: Curva di apprendimento (loss) dell'MLP.

Tabella 9: Report di classificazione — MLP (accuratezza **53.99%**).

	Basso	Medio	Alto	Macro	Pesata
Precision	0.54	0.53	0.56	0.54	0.54
Recall	0.57	0.48	0.59	0.55	0.54
F1	0.55	0.50	0.57	0.54	0.54

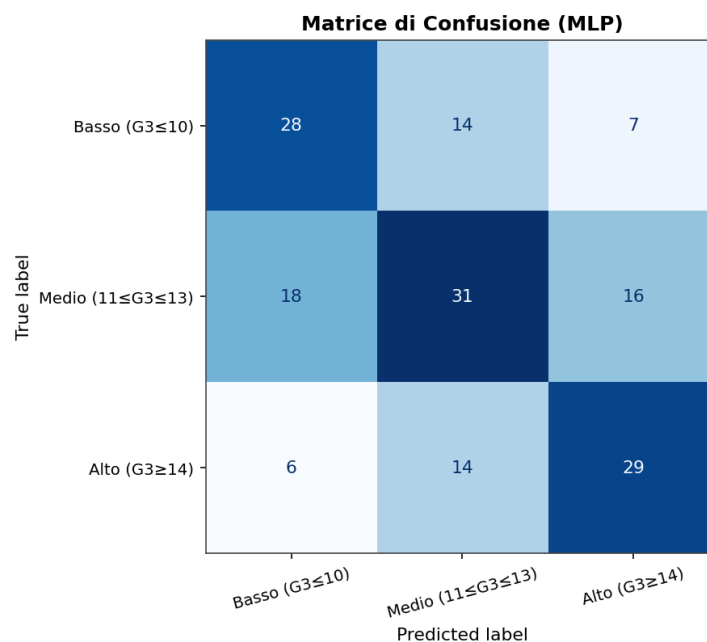


Figura 23: Matrice di confusione — MLP.

9 Confronto dei Modelli

Per valutare l'effetto della **riduzione della dimensionalità** sulle prestazioni, ogni classificatore viene addestrato e valutato su tre rappresentazioni dei dati:

- **Originale** — le 43 feature standardizzate;
- **PCA (95%)** — le componenti principali che spiegano il 95% della varianza (non supervisionata): 34;
- **MDA (2D)** — le 2 componenti discriminanti di Fisher (supervisionata).

Il confronto mostra se uno spazio compatto preserva (o migliora) l'accuratezza rispetto allo spazio completo, e mette a confronto una proiezione *non supervisionata* (PCA) con una *supervisionata* (MDA).

Nota. La proiezione **MDA (2D)** è *supervisionata*: i suoi assi sono costruiti usando le etichette del training per massimizzare la separabilità. Il confronto PCA *vs* MDA non è quindi tra due riduzioni equivalenti, e l'eventuale guadagno dei classificatori sullo spazio MDA va letto tenendo conto che parte del lavoro discriminante è già stato svolto da Fisher.

Tabella 10: Accuratezza (%) sul test set per modello e spazio di rappresentazione.

Modello	Originale	PCA (95%)	MDA (2D)
MDA/LDA	55.83	54.60	55.83
QDA	40.49	53.37	57.06
SVM	54.60	52.15	57.06
K-NN	53.37	49.08	59.51
MLP	53.99	49.69	58.28
Baseline (classe maggioritaria): 39.75%			

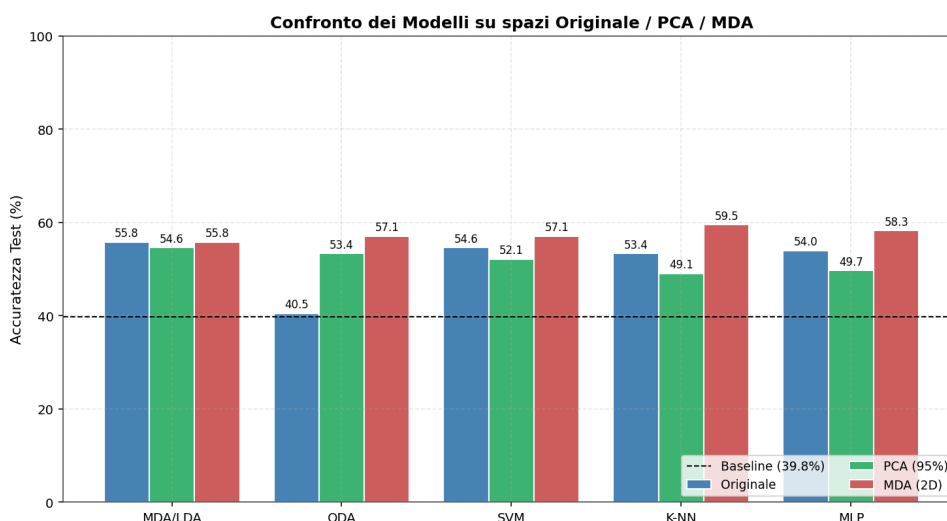


Figura 24: Confronto dei modelli su spazi Originale / PCA / MDA.

La combinazione migliore è il **K-NN su spazio MDA (2D)** con **59.51%**.

9.1 Bilanciamento delle Classi e Metriche Robuste

La discretizzazione a **terzili** (§4) rende le tre fasce di numerosità simile (~30/40/30%), eliminando alla radice gran parte dello sbilanciamento. Resta comunque buona pratica affiancare

all'accuratezza la **macro-F1** (media non pesata delle F1 di classe: dà lo stesso peso a ogni fascia) per verificare che nessuna classe venga trascurata. Un riequilibrio esplicito (ad es. `class_weight='balanced'` per la SVM) porta un effetto marginale su un target già bilanciato: conferma indiretta che il bilanciamento *a monte* (terzili) è la soluzione più pulita rispetto al riequilibrio *a valle* sui pesi.

Tabella 11: Accuratezza vs macro-F1 sul test set (spazio originale).

Modello	Accuratezza	Macro-F1
MDA/LDA	55.83%	0.554
QDA	40.49%	0.360
SVM	54.60%	0.548
K-NN	53.37%	0.521
MLP	53.99%	0.543

9.2 Visualizzazione degli Errori sulla Proiezione

Sull'esempio della miglior SVM, coloriamo i punti del **test set** sulla proiezione 2D della PCA distinguendo le classificazioni **corrette** da quelle **errate** (cerchi neri). È la lettura qualitativa complementare alla matrice di confusione: gli errori si concentrano dove le nuvole di classe si sovrappongono — la fascia “Buono” centrale — confermando che la frontiera è intrinsecamente ambigua, non un difetto del singolo modello.

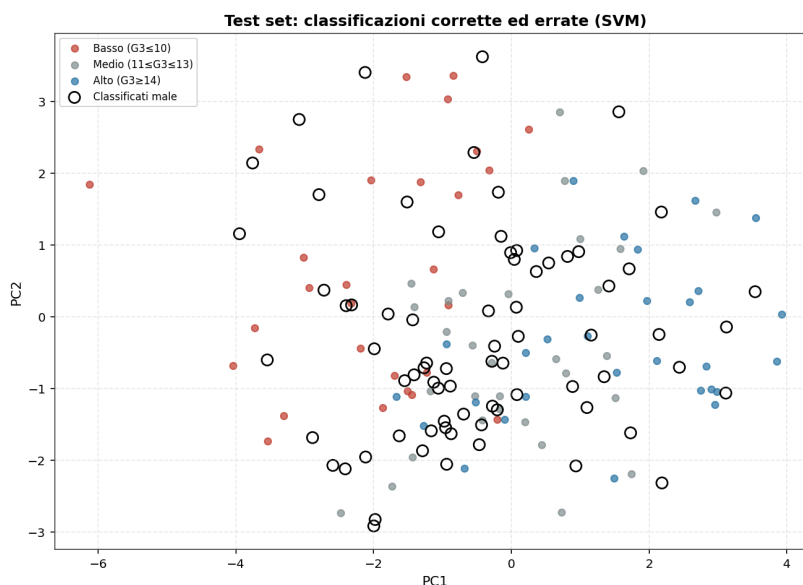


Figura 25: Test set: classificazioni corrette ed errate della miglior SVM.

Gli errori della miglior SVM sono **74/163** (45.4%), concentrati sulla fascia centrale “Medio”.

9.3 Validazione robusta: Cross-Validation (k-fold)

Il confronto delle sezioni precedenti poggia su un **singolo split** 75/25: con ~ 163 campioni di test l'errore campionario dell'accuratezza è dell'ordine di $\pm 7-8$ punti percentuali — l'incertezza

statistica tipica di una percentuale stimata su ~ 163 osservazioni — entro cui le differenze tra i modelli di testa **non sono distinguibili**. Per una stima più stabile valutiamo ogni modello con **5-fold stratificata** (5 fit), riportando media, deviazione standard e intervallo di confidenza al 95%. Lo StandardScaler è incluso nella Pipeline, quindi viene ri-stimato su ogni fold di addestramento: nessun *data leakage*.

Gli iperparametri (C, gamma per la SVM; K per il K-NN) sono quelli selezionati nelle sezioni precedenti; LDA/QDA non hanno iperparametri rilevanti e l'MLP usa l'architettura fissata. La cross-validation misura quindi la **stabilità** della stima e la sua incertezza, non un nuovo tuning. (Nota: con soli 5 fold l'intervallo di confidenza è indicativo.)

Tabella 12: Validazione robusta: 5-fold CV stratificata (spazio originale standardizzato). Modelli ordinati per accuratezza media; IC 95% dalla *t* di Student sui 5 fold.

Modello	Accuratezza media (%)	Dev. std (%)	IC 95% (%)
SVM	56.85	3.98	[51.3, 62.4]
MDA/LDA	56.69	4.81	[50.0, 63.4]
MLP	55.00	2.62	[51.4, 58.6]
K-NN	50.23	2.09	[47.3, 53.1]
QDA	46.38	4.28	[40.4, 52.3]
<i>Baseline (classe magg.)</i>		39.75	

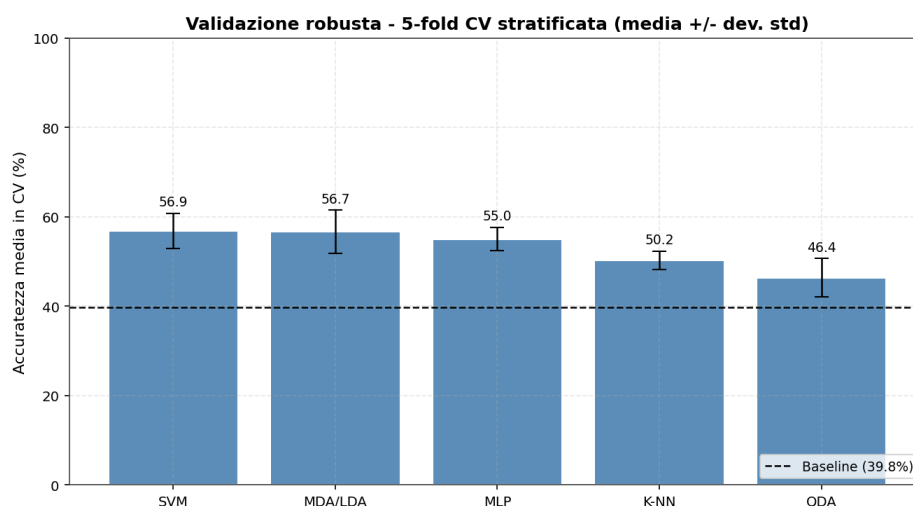


Figura 26: Validazione robusta: 5-fold CV stratificata (media $\pm$ dev. std tra i fold).

Commento al confronto finale. Sullo split singolo il K-NN su MDA (2D) è il più accurato, ma i primi classificati distano pochi punti e i loro **intervalli di confidenza al 95% si sovrappongono** — i modelli di testa sono statisticamente **indistinguibili**. Il limite è la struttura dei dati (fasce 2D fortemente sovrapposte, soprattutto la classe centrale “Medio”), non l’algoritmo: i modelli flessibili (SVM-RBF, MLP) non possono sfruttare confini complessi e si allineano ai lineari (MDA/LDA), la QDA resta indietro, il K-NN recupera nettamente una volta proiettato sulla MDA.

L’analisi completa — effetto della dimensionalità (PCA vs MDA), ruolo della discretizzazione a terzi e scelta pratica del modello — è sviluppata nelle **Conclusioni (§10)**.

10 Conclusioni

Combinando l'analisi esplorativa (**PCA/MDA**) con i risultati quantitativi dei classificatori si possono trarre conclusioni sulla previsione del rendimento, leggendo i numeri nella tabella di confronto del §9 e sempre rispetto alla **baseline** (~39.8%).

Quale modello vince e perché. In generale, su questo dataset *rumoroso* i confini **lineari** (MDA/LDA, SVM lineare) e l'SVM con kernel RBF si comportano in modo simile, perché le classi — viste nelle proiezioni 2D — sono **fortemente sovrapposte** e non esiste una frontiera netta da catturare: nessun modello può andare molto oltre la struttura realmente presente nei dati. L'SVM trae forza dalla *Grid Search* con 5-fold CV (sceglie margine e kernel in modo robusto); la MDA/LDA resta competitiva pur essendo il modello più semplice e interpretabile. La **cross-validation** (§9.3) conferma però che gli intervalli di confidenza al 95% dei modelli di testa si **sovrappongono**: le differenze osservate sul singolo split rientrano nel rumore campionario.

Perché alcuni modelli falliscono.

- la **QDA** stima una covarianza per ogni classe e, su fasce con poche centinaia di campioni in 43 dimensioni, queste stime sono instabili → **overfitting** e crollo dell'accuratezza rispetto alla LDA; vi contribuisce anche la ridondanza tra le variabili categoriche codificate (feature correlate);
- il **K-NN** soffre la **maledizione della dimensionalità**: in 43 dimensioni le distanze euclidee si concentrano e perdono potere discriminante, per questo è il più penalizzato sullo spazio originale;
- la **SVM a margine quasi-rigido** ($C \rightarrow \infty$) pretende di separare perfettamente dati non separabili: il margine si restringe leggermente ($2/\|w\|$ 3.503 vs 3.882), ma su dati non separabili anche un C enorme non riesce ad azzerare gli errori di training, quindi qui il divario di accuratezza train/test resta contenuto rispetto al margine morbido — il segnale di overfitting è nel margine, non (in questo caso) nell'accuratezza sul test;
- l'**MLP**, con ~650 campioni e decine di predittori binari/ordinali, è esposto all'overfitting (mitigato da α ed early stopping) e rende al meglio solo dopo la riduzione di dimensionalità.

Effetto della dimensionalità. Il confronto su spazi Originale / PCA / MDA mostra due fatti opposti. La **PCA** (non supervisionata, 34 PC su 43 per il 95%) **non** migliora l'accuratezza: la riduce di qualche punto per tutti i modelli, SVM compresa, comprimendo il solo rumore senza creare potere discriminante. La **MDA (2D)**, supervisionata, **migliora i modelli che soffrono l'alta dimensione** (K-NN e MLP in particolare), perché concentra in 2 assi proprio la separazione tra classi: è la prova empirica della differenza concettuale tra riduzione non supervisionata e supervisionata.

Effetto della discretizzazione di G3. Trasformare un voto continuo (0–20) in 3 fasce è una semplificazione che **distrugge informazione ordinale** ai bordi delle fasce (uno studente con $G3 = 13$ e uno con $G3 = 14$ finiscono in classi diverse pur essendo quasi identici): parte dell'errore dei classificatori è *intrinseca* a questa scelta. Il **binning a terzili** adottato è però il compromesso migliore: rende le classi di numerosità confrontabile (~30/40/30%, baseline ≈ 39.8%), così l'accuratezza è una metrica più equilibrata — pur non del tutto attendibile da sola, per cui la affianchiamo alla macro-F1 — e il confronto con la baseline è più stringente; soglie più sbilanciate avrebbero gonfiato l'accuratezza pur lasciando intere fasce mai predette

(come mostra la macro-F1). La sufficienza ufficiale $G3 = 10$ resta catturata a parte dal target binario della FDA.

PCA o MDA: quale preserva meglio la separabilità. A parità di dimensioni la **MDA** è nettamente superiore alla PCA nel preservare la separabilità delle fasce: con **2 soli assi supervisionati** ogni classificatore pareggia o supera lo spazio originale a 43 feature (K-NN +6.1, MLP +4.3 punti), mentre la PCA (34 PC per il 95%) non migliora l'accuratezza. La ragione è concettuale: la PCA massimizza la varianza *totale* ignorando le etichette, la MDA la separazione *tra classi*. La PCA resta utile per denoising e visualizzazione non supervisionata, ma per discriminare le fasce di rendimento la MDA è la scelta migliore.

Tabella finale di confronto. La Tabella 13 riporta la miglior accuratezza di ciascun metodo sul Test Set.

Tabella 13: Tabella finale di confronto: miglior accuratezza per metodo sul Test Set.

Metodo	Accuratezza	Spazio	Nota
K-NN	59.51%	MDA (2D)	massimo, ma recupera solo dopo la riduzione
MLP	58.28%	MDA (2D)	secondo, vantaggio entro il rumore
QDA	57.06%	MDA (2D)	recupera in 2D; su Originale crolla a 40.49% (overfitting covarianze)
SVM	57.06%	MDA (2D)	massimo su MDA (pari a QDA); 54.60% su Originale, 52.15% su PCA
FDA/LDA	55.83%	Originale	migliore interpretabilità

I quattro modelli principali stanno entro **~3.7 punti**: nessuno domina. Su singolo split queste cifre vanno però prese con cautela ($\pm 7-8$ punti di errore campionario su ~ 163 test): la **cross-validation** (§9.3) mostra IC 95% sovrapposti per il gruppo di testa, quindi la classifica puntuale **non è statisticamente significativa**. Il K-NN raggiunge il massimo solo grazie alla proiezione MDA — sullo spazio originale è tra i peggiori — e con ~ 650 campioni il suo margine è nei limiti del rumore: **non offre vantaggi pratici decisivi**. La SVM è la più robusta e consistente tra le rappresentazioni. La FDA/LDA, a parità di accuratezza, offre la **migliore interpretabilità** (modello lineare, proiezione 2D leggibile): per un sistema di *early warning* scolastico è spesso la scelta preferibile.

Quadro complessivo, limiti del dataset e sviluppi futuri. Tra i modelli *supervisionati* il quadro è compatto: prestazioni simili, con la scelta pratica orientata da interpretabilità (FDA/LDA) e robustezza (SVM) più che dalla sola accuratezza. I risultati vanno letti alla luce dei **limiti del dataset**:

1. solo **649 campioni**, troppo pochi perché modelli complessi (MLP, QDA) esprimano un vantaggio senza overfittare;
2. avendo escluso G1/G2 restano predittori **socio-comportamentali** debolmente legati al voto, che impongono un tetto basso all'accuratezza;
3. la **discretizzazione** di G3 distrugge l'informazione ordinale;
4. i dati provengono da **una sola materia e due scuole**, limitando la generalizzabilità.

Sviluppi futuri: trattare G3 come variabile continua (**regressione**) o con **classificazione ordinale**; **selezione/ingegnerizzazione delle feature** per ridurre il rumore; modelli **ensemble** (Random Forest, Gradient Boosting), spesso robusti su dati tabellari piccoli; **nested cross-validation** per stime di generalizzazione piu' solide; un task distinto con G1/G2 inclusi per la previsione *in itinere*; e l'estensione della stessa pipeline al **corso di Matematica** (395 studenti) o al dataset **fuso** dei 382 studenti comuni ai due corsi, per verificare la generalità delle conclusioni.

Sintesi. Avendo escluso G1/G2 (target leakage), il rendimento va previsto da soli predittori socio-comportamentali, intrinsecamente rumorosi: le accuratezze ($\approx$ baseline+ 10–20 punti) vanno lette in quest'ottica. Come suggerito dai *loadings* della PCA, il successo accademico dipende da un ecosistema di cause (background familiare, stile di vita, storico di bocciature). Un sistema di *early warning* basato su MDA/LDA o SVM, valutato sulla **macro-F1** e non sulla sola accuratezza, potrebbe individuare per tempo pattern sfavorevoli.

11 Bibliografia

1. P. Cortez, A. Silva. *Using Data Mining to Predict Secondary School Student Performance*. Proc. FUBUTEC 2008, pp. 5–12, Porto, 2008.
2. D. Dua, C. Graff. *UCI Machine Learning Repository — Student Performance Data Set*. University of California, Irvine, 2019.
<https://archive.ics.uci.edu/dataset/320/student+performance>
3. C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
4. T. Hastie, R. Tibshirani, J. Friedman. *The Elements of Statistical Learning*. 2nd ed., Springer, 2009.
5. R. O. Duda, P. E. Hart, D. G. Stork. *Pattern Classification*. 2nd ed., Wiley, 2001.
6. F. Pedregosa et al. *Scikit-learn: Machine Learning in Python*. JMLR, 12:2825–2830, 2011.